

Computer Networks

The Ultimate Guide

OSI · TCP/IP · IP Addressing · Routing · DNS · HTTP/S
TLS · TCP · UDP · QUIC · Sockets · BGP · Security
CDNs · Load Balancing · Wireless · SDN · Interview Prep

Created by → **Sahil Raj**

Codeforces → [After_Dark](#)

Contents

1	The OSI and TCP/IP Models	7
1.1	Why We Need Layered Models	7
1.2	The OSI Model (7 Layers)	7
1.2.1	Layer 1: Physical Layer	8
1.2.2	Layer 2: Data Link Layer	8
1.2.3	Layer 3: Network Layer	9
1.2.4	Layer 4: Transport Layer	9
1.2.5	Layers 5, 6, 7: Session, Presentation, Application	9
1.3	The TCP/IP Model (4 or 5 Layers)	10
2	IP Addressing, Subnetting, and IPv6	11
2.1	IPv4 Address Structure	11
2.1.1	Classful Addressing (Historical — Pre-CIDR)	11
2.1.2	Special IPv4 Addresses	12
2.2	CIDR and Subnetting	12
2.2.1	CIDR Quick Reference Table	13
2.2.2	Subnetting: Worked Problems	13
2.3	IPv6: The 128-bit Address Space	14
2.3.1	IPv6 Address Format and Notation	14
2.3.2	IPv6 Address Types	14
2.3.3	IPv6 Header Structure	15
2.4	ARP: Address Resolution Protocol	15
2.5	NAT: Network Address Translation	16
3	TCP: The Reliable Transport Protocol	17
3.1	TCP Segment Header — Every Field Explained	17
3.1.1	TCP Control Flags — What Each Does	17
3.2	TCP Connection Lifecycle	18
3.2.1	Three-Way Handshake: Establishing a Connection	18
3.2.2	Four-Way Teardown: Closing a Connection	18
3.2.3	TCP State Machine	19
3.3	TCP Reliability Mechanisms	19
3.3.1	Sequence Numbers, ACKs, and Byte Streams	20
3.3.2	RTO Calculation (RFC 6298 — Jacobson/Karels Algorithm)	20
3.3.3	Fast Retransmit	20
3.3.4	Selective Acknowledgements (SACK)	20
3.3.5	D-SACK (Duplicate SACK, RFC 2883)	20
3.4	TCP Flow Control: Sliding Window	20
3.5	TCP Congestion Control	21
3.5.1	TCP Reno / Tahoe Phases	22
3.5.2	TCP CUBIC (Linux Default Since 2.6.19)	22
3.5.3	BBR (Bottleneck Bandwidth and Round-trip propagation time)	22

3.6	TCP Options Reference	23
4	UDP and Lightweight Transport Protocols	25
4.1	UDP Header — Minimal Design	25
4.2	When UDP is the Right Choice	25
4.3	QUIC: The Future of Transport	26
4.4	SCTP: Stream Control Transmission Protocol	27
5	DNS: The Internet’s Directory Service	29
5.1	DNS Architecture and Hierarchy	29
5.2	Full DNS Resolution Walk-Through	29
5.3	DNS Record Types — Complete Reference	30
5.4	DNS Caching, TTL, and Negative Caching	30
5.5	DHCP: Dynamic Host Configuration Protocol	31
5.6	DNS Security: Attacks and Defences	32
6	HTTP, HTTPS, and the Application Protocol Stack	33
6.1	HTTP Evolution: From 0.9 to 3	33
6.2	HTTP Request and Response Structure	34
6.3	HTTP Methods	34
6.4	HTTP Status Codes — Complete Reference	36
6.5	HTTP Caching	37
6.6	Cookies, Sessions, and Authentication	37
6.7	CORS: Cross-Origin Resource Sharing	38
7	TLS/SSL: Transport Layer Security	39
7.1	What TLS Provides	39
7.2	TLS 1.3 Handshake (1-RTT)	40
7.2.1	Key Exchange: ECDHE (Elliptic Curve Diffie-Hellman Ephemeral)	40
7.2.2	Certificate Chain of Trust	41
7.2.3	TLS Cipher Suites	41
7.3	TLS 1.3 vs TLS 1.2	42
8	Routing and Routing Protocols	43
8.1	How Packet Forwarding Works	43
8.2	Routing Protocols Classification	44
8.2.1	RIP: Routing Information Protocol (Historical)	44
8.2.2	OSPF: Open Shortest Path First	44
8.2.3	BGP: Border Gateway Protocol	44
8.3	VLANs: Virtual LANs	45
8.4	MPLS: Multiprotocol Label Switching	46
9	Network Security	47
9.1	Common Network Attacks	47
9.1.1	TCP SYN Flood	47
9.1.2	DDoS: Distributed Denial of Service	48
9.1.3	MITM Attacks	48
9.2	Firewalls	49
9.3	VPN: Virtual Private Networks	50
9.4	Network Scanning and Intrusion Detection	50
10	Socket Programming and Network I/O	51
10.1	The Socket API: Fundamental Concepts	51
10.2	Key Socket Options Reference	54
10.3	I/O Multiplexing: Handling Thousands of Connections	54

10.3.1	The C10K and C10M Problem	54
10.3.2	select() and poll()	54
10.3.3	epoll: The Linux Standard for High-Performance I/O	56
10.3.4	io_uring (Linux 5.1+, 2019)	57
10.3.5	The Reactor and Proactor Patterns	57
10.4	Non-blocking Sockets and Common Pitfalls	58
10.5	WebSocket Protocol	58
11	Application Layer Protocols and Architectures	61
11.1	Email Protocols: SMTP, POP3, IMAP	61
11.2	SSH: Secure Shell	61
11.3	FTP and Its Quirks	62
11.4	REST and gRPC	63
11.5	CDNs: Content Delivery Networks	64
11.6	Load Balancing	64
12	Wireless Networks	67
12.1	IEEE 802.11 Wi-Fi Standards	67
12.1.1	CSMA/CA: Collision Avoidance	67
12.1.2	Wi-Fi 6 (802.11ax) Key Improvements	67
12.2	Wi-Fi Security	68
13	Advanced Topics: SDN, Cloud Networking, and Modern Architectures	69
13.1	Software Defined Networking (SDN)	69
13.1.1	OpenFlow	69
13.2	Container and Cloud Networking	69
13.2.1	Network Namespaces (Linux)	69
13.2.2	Docker Networking	70
13.2.3	Kubernetes Networking	70
13.3	eBPF: The Revolution in Network Programmability	71
14	Network Diagnostic Tools	73
14.1	Essential Linux Networking Commands	74
14.2	Reading tcpdump Output	75
15	Networking Interview Masterclass	77
15.1	System Design and Networking Questions	77
15.2	TCP/IP Deep Questions	79
15.3	Security Interview Questions	80

Chapter 1

The OSI and TCP/IP Models

1.1 Why We Need Layered Models

Networking is enormously complex. A single web request involves your Wi-Fi radio, your home router, your ISP's network, undersea fibre-optic cables, multiple backbone routers, cloud load balancers, and finally a server rack — all in under 200 milliseconds. Without structure, this would be unimaginable to engineer, debug, or modify.

Layered models tame this complexity by dividing the problem into a vertical stack of well-defined, independent layers. Each layer:

- Provides a **service** to the layer above it.
- Uses the **service** of the layer below it.
- Communicates with its **peer layer** on the remote host using a protocol.
- Is **transparent** to what's inside other layers (encapsulation principle).

This means you can swap Ethernet for Wi-Fi (Layer 2) without changing TCP (Layer 4) or HTTP (Layer 7) at all. This **modularity** is the defining engineering achievement of networking.

Concept / Pattern

The Core Principle: Encapsulation and Decapsulation

Sending (top-down): Each layer **wraps** (encapsulates) the data from the layer above with its own **header** (and sometimes a **trailer**). The resulting unit is called the layer's PDU (Protocol Data Unit).

Receiving (bottom-up): Each layer **strips** its own header/trailer and passes the payload up. The peer layer on the other side is the only entity that “reads” that header.

Flow example:

1. Application data (HTTP message, e.g., “GET /index.html”)
2. → TCP adds segment header → **TCP Segment**
3. → IP adds packet header → **IP Packet**
4. → Ethernet adds frame header + trailer → **Ethernet Frame**
5. → Physical layer transmits as bits/signals on the wire

1.2 The OSI Model (7 Layers)

The **Open Systems Interconnection** model (ISO/IEC 7498-1, 1984) is the canonical *reference model*. Real protocols don't map perfectly to it, but it is the universal language for discussing networking.

Deep Dive / Detail

#	Layer	PDU	Key Protocols	Devices
7	Application	Message	HTTP, FTP, SMTP, DNS, SSH	Host
6	Presentation	Message	TLS/SSL, JPEG, ASCII, Compression	Host
5	Session	Message	RPC, NetBIOS, SMB, SIP	Host
4	Transport	Segment/Datagram	TCP, UDP, SCTP	Host
3	Network	Packet	IP, ICMP, OSPF, BGP, ARP	Router
2	Data Link	Frame	Ethernet, Wi-Fi (802.11), PPP	Switch, Bridge
1	Physical	Bit/Symbol	Copper, Fibre, Radio, RS-232	Hub, NIC, Repeater

Mnemonic (7 down to 1): *All People Seem To Need Data Processing*

Mnemonic (1 up to 7): *Please Do Not Throw Sausage Pizza Away*

1.2.1 Layer 1: Physical Layer

Transmits raw **bits** (0s and 1s) over a physical medium. Concerned with:

- **Bit encoding:** How bits are represented — NRZ (Non-Return to Zero), Manchester encoding (Ethernet 10Mbps), MLT-3 (Fast Ethernet), PAM4 (100G+).
- **Modulation:** How data is modulated onto a carrier — QAM-256, OFDM (Wi-Fi), DMT (DSL).
- **Signal timing and synchronisation:** Clocking, preambles.
- **Physical topology:** Bus, Ring, Star, Mesh.
- **Medium:** Twisted pair (Cat5e/Cat6/Cat6a), coaxial, single-mode fibre, multi-mode fibre, radio waves.
- **Devices:** NIC (Network Interface Card), hub (dumb repeater), modem, repeater, transceiver.

Deep Dive / Detail

Cable Categories and Their Specs

Cable	Speed	Bandwidth	Max distance	Use
Cat5e	1 Gbps	100 MHz	100m	Office LAN
Cat6	10 Gbps	250 MHz	55m (10G), 100m (1G)	Modern office LAN
Cat6a	10 Gbps	500 MHz	100m	Data centres
Cat7	10 Gbps	600 MHz	100m	Shielded; industrial
Cat8	40 Gbps	2000 MHz	30m	Data centre top-of-rack
SMF (single-mode)	100 Gbps+	—	100+ km	Long-haul, WAN
MMF (multi-mode)	10–100 Gbps	—	300–550m	Data centre

1.2.2 Layer 2: Data Link Layer

Provides reliable **node-to-node** (hop-by-hop) delivery on the *same network segment*. Uses **MAC addresses** (48-bit, globally unique, burned into NIC hardware, also called hardware address or physical address).

Sub-layers:

- **LLC (Logical Link Control):** Error detection (CRC), flow control. IEEE 802.2.
- **MAC (Media Access Control):** Controls access to the shared medium (CSMA/CD for Ethernet, CSMA/CA for Wi-Fi).

Ethernet frame structure (IEEE 802.3):

Preamble (7B)	SFD (1B)	Dst MAC (6B)	Src MAC (6B)	EtherType (2B)	Payload (46–1500B)	FCS (4B)
---------------	----------	--------------	--------------	----------------	--------------------	----------

- **Preamble:** 7 bytes of 10101010... for synchronisation (clock recovery).
- **SFD (Start Frame Delimiter):** 10101011 signals start of frame.
- **EtherType:** 0x0800 = IPv4, 0x0806 = ARP, 0x86DD = IPv6, 0x8100 = VLAN (802.1Q), 0x8847 = MPLS.
- **FCS (Frame Check Sequence):** CRC-32 for error *detection* (not correction). Errored frames are silently dropped.
- **Minimum frame size:** 64 bytes (to detect collisions on 10Mbps Ethernet with 100m cable). Frames < 64B are “runt” and dropped.
- **MTU:** Maximum Transmission Unit = 1500 bytes of payload. Can be changed (jumbo frames = 9000B in data centres).

Concept / Pattern**MAC Address Structure**

AA:BB:CC:DD:EE:FF — 6 bytes, written in hex.

- **OUI (Organisationally Unique Identifier):** First 3 bytes (AA:BB:CC) assigned by IEEE to the manufacturer.
- **NIC specific:** Last 3 bytes assigned by manufacturer.
- **Bit 1 of first byte:** 0 = unicast, 1 = multicast.
- **Bit 2 of first byte:** 0 = globally unique, 1 = locally administered.
- **Broadcast:** FF:FF:FF:FF:FF:FF. All stations on segment receive.
- **IPv4 multicast:** 01:00:5E:xx:xx:xx (lower 23 bits from IP multicast group).

1.2.3 Layer 3: Network Layer

Provides **end-to-end logical addressing** (IP addresses) and **routing** of packets across multiple networks. Key distinction: Layer 2 handles one hop at a time; Layer 3 handles the complete source-to-destination journey.

- **Routers** operate here. They strip and re-add Layer 2 frames at each hop, but the IP packet remains intact (except TTL decrement) from source to destination.
- Key protocols: **IP** (addressing + forwarding), **ICMP** (control messages), **ARP** (IP-to-MAC resolution, technically Layer 2.5), **IGMP** (multicast group management), **OSPF**, **BGP** (routing).

1.2.4 Layer 4: Transport Layer

Provides **process-to-process** (end-to-end) communication using **port numbers** (0–65535). The two dominant protocols:

- **TCP:** Reliable, ordered, connection-oriented. Flow control, congestion control.
- **UDP:** Unreliable, unordered, connectionless. Minimal overhead.
- **SCTP:** Stream Control Transmission Protocol — multi-homing, multi-streaming.

1.2.5 Layers 5, 6, 7: Session, Presentation, Application

These are often collapsed into the “Application layer” in the TCP/IP model:

- **Layer 5 (Session):** Manages sessions (dialogues): establish, maintain, terminate. RPC, NetBIOS, SIP.
- **Layer 6 (Presentation):** Data translation, encryption, compression. TLS fits here conceptually; JPEG, ASCII, Unicode encode data.
- **Layer 7 (Application):** User-facing protocols: HTTP, FTP, SMTP, DNS, SSH, SNMP.

1.3 The TCP/IP Model (4 or 5 Layers)

The practical model the real Internet runs on, developed by DARPA (1970s):

Concept / Pattern

TCP/IP Layer	OSI Equivalent	Key Protocols
Application	5, 6, 7	HTTP/S, DNS, FTP, SMTP, SSH, TLS, SNMP, NTP
Transport	4	TCP, UDP, QUIC, SCTP
Internet	3	IPv4, IPv6, ICMP, ICMPv6, ARP, OSPF, BGP
Network Access	1, 2	Ethernet, Wi-Fi (802.11), PPP, DSL, ATM

Note: Some models add a 5th “Physical” layer, separating Physical from Data Link.

Interview Corner

Classic OSI interview questions

Q: At which layer does a switch operate? A router?

Switch: Layer 2 (forwards based on MAC address table; learns by observing source MACs). Router: Layer 3 (forwards based on IP routing table using LPM). A *Layer 3 switch* does both: routes between VLANs using IP while switching within VLANs using MAC.

Q: What is the difference between a hub and a switch?

A hub (Layer 1) simply repeats all incoming bits out all ports — same collision domain. A switch (Layer 2) learns which MAC addresses are on which port and forwards frames *only* to the correct destination port, reducing collisions and improving efficiency. Hubs are obsolete.

Q: Why does a router decrement the TTL (Time-To-Live) field?

Each router decrements TTL by 1. If TTL reaches 0, the router drops the packet and sends an ICMP “Time Exceeded” message to the source. This prevents packets from looping forever in case of a routing loop. `traceroute` exploits this: it sends packets with TTL=1, 2, 3... to map each hop.

Q: What is a collision domain vs. a broadcast domain?

Collision domain: Devices that can cause collisions with each other (CSMA/CD). Hubs extend collision domains; switches *break* them (each port is its own collision domain). **Broadcast domain:** Devices that receive each other’s L2 broadcasts (FF:FF:FF:FF:FF:FF). Switches extend broadcast domains; routers *break* them. VLANs break broadcast domains at Layer 2.

Chapter 2

IP Addressing, Subnetting, and IPv6

2.1 IPv4 Address Structure

An IPv4 address is 32 bits, written as 4 dotted-decimal octets: 192.168.1.100.

Concept / Pattern

Binary Representation

192.168.1.100 in binary:

192 .168 .1 .100

11000000 . 10101000 . 00000001 . 01100100

Total possible IPv4 addresses: $2^{32} = 4,294,967,296 \approx 4.3$ billion.

An IPv4 address has two parts:

- **Network portion:** Identifies the network (all hosts on same network have same network bits).
- **Host portion:** Identifies the specific host within the network.

The **subnet mask** separates them: 1s in the mask = network bits; 0s = host bits.

2.1.1 Classful Addressing (Historical — Pre-CIDR)

Class	First bits	First octet range	Default mask	Networks	Hosts/net
A	0	1–126	/8 (255.0.0.0)	128	16,777,214
B	10	128–191	/16 (255.255.0.0)	16,384	65,534
C	110	192–223	/24 (255.255.255.0)	2,097,152	254
D	1110	224–239	—	Multicast groups	—
E	1111	240–255	—	Reserved (experimental)	—

Common Mistake / Gotcha

Classful addressing was extremely wasteful. A company needing 260 hosts got a Class B (/16, 65,534 hosts), wasting 65,274 addresses. **CIDR** replaced classful addressing in 1993.

2.1.2 Special IPv4 Addresses

Address/Range	Purpose and Notes
127.0.0.0/8	Loopback. Packets never leave the host. 127.0.0.1 = “localhost”. All 127.x.x.x loop.
0.0.0.0/8	“This host on this network” OR “any address”. Used in server <code>bind()</code> .
255.255.255.255	Limited broadcast. All hosts on local network. Not routed.
10.0.0.0/8	Private (RFC 1918). ~16M addresses. Not routed on Internet.
172.16.0.0/12	Private (RFC 1918). 172.16–172.31.x.x. ~1M addresses.
192.168.0.0/16	Private (RFC 1918). Most common home network range.
169.254.0.0/16	Link-local / APIPA. Auto-assigned when DHCP fails. OS-assigned.
224.0.0.0/4	Multicast (Class D). 224.0.0.1 = all hosts, 224.0.0.2 = all routers.
100.64.0.0/10	Shared Address Space (RFC 6598). Used by ISPs for CGNAT.
198.18.0.0/15	Benchmarking (RFC 2544). Network equipment testing.
192.0.2.0/24	TEST-NET-1 (RFC 5737). Documentation and examples.
198.51.100.0/24	TEST-NET-2. Documentation.
203.0.113.0/24	TEST-NET-3. Documentation.

2.2 CIDR and Subnetting

CIDR (Classless Inter-Domain Routing) (RFC 4632, 1993) replaced the classful system. An address block is written as IP/prefix-length (the “slash notation” or “CIDR notation”).

Concept / Pattern

Understanding the Subnet Mask Step-by-Step

For 192.168.1.0/24:

- **Prefix (network) bits:** 24 bits, all 1s in the mask: 255.255.255.0
- **Host bits:** $32 - 24 = 8$ bits.
- **Network address:** All host bits = 0: 192.168.1.0
- **Broadcast address:** All host bits = 1: 192.168.1.255
- **Usable host addresses:** $2^8 - 2 = 254$ (subtract network + broadcast).
- **Range of usable hosts:** 192.168.1.1 through 192.168.1.254

To find the network address from any IP: Bitwise AND the IP with the mask.

192.168.1.100 && 255.255.255.0 = 192.168.1.0 ✓

2.2.1 CIDR Quick Reference Table

CIDR	Subnet Mask	Total IPs	Usable Hosts	Block	Typical Use
/8	255.0.0.0	16,777,216	16,777,214	16M	ISP / large enterprise
/16	255.255.0.0	65,536	65,534	64K	Large campus LAN
/20	255.255.240.0	4,096	4,094	4K	Large department
/22	255.255.252.0	1,024	1,022	1K	Medium network
/24	255.255.255.0	256	254	256	Typical office subnet
/25	255.255.255.128	128	126	128	Split of a /24
/26	255.255.255.192	64	62	64	Quarter of a /24
/27	255.255.255.224	32	30	32	Small team LAN
/28	255.255.255.240	16	14	16	Tiny subnet
/29	255.255.255.248	8	6	8	Very small subnet
/30	255.255.255.252	4	2	4	Point-to-point WAN link
/31	255.255.255.254	2	2 (RFC 3021)	2	P2P link (no broadcast)
/32	255.255.255.255	1	1 host route	1	Single host or loopback

2.2.2 Subnetting: Worked Problems

Problem 1: Divide 192.168.10.0/24 into **6 subnets**, each needing at least 20 hosts.

1. **How many bits needed for 6 subnets?** $\lceil \log_2 6 \rceil = 3$ bits. Borrow 3 bits from host portion.
2. **New prefix:** $24 + 3 = /27$.
3. **Hosts per subnet:** $2^{32-27} - 2 = 2^5 - 2 = 30$. ✓ (satisfies ≥ 20)
4. **Block size:** $2^5 = 32$. Subnets increment by 32.

Subnet	Network	First Host	Last Host	Broadcast
1	192.168.10.0/27	.1	.30	.31
2	192.168.10.32/27	.33	.62	.63
3	192.168.10.64/27	.65	.94	.95
4	192.168.10.96/27	.97	.126	.127
5	192.168.10.128/27	.129	.158	.159
6	192.168.10.160/27	.161	.190	.191

Problem 2: Given IP 172.20.35.189/20, find network address, broadcast, and number of usable hosts.

1. **Mask:** /20 $\Rightarrow$ 255.255.240.0. Third octet mask = 11110000 = 240.
2. **Network address:** AND with mask.
Third octet: 35 in binary = 00100011. AND with 11110000 = 00100000 = 32.
Network = 172.20.32.0
3. **Host bits:** $32 - 20 = 12$ bits.
4. **Broadcast:** Set all 12 host bits to 1.
Third octet: 00101111 = 47. Fourth octet = 255. Broadcast = 172.20.47.255
5. **Usable hosts:** $2^{12} - 2 = 4094$.
6. **Is .189 in this subnet?** .189 is between .0 and .255, within 172.20.32–47 range. Yes!

Problem 3 (VLSM): You have 10.0.0.0/24. Allocate subnets for: Dept A (60 hosts), Dept B (30 hosts), two P2P links (2 hosts each).

1. Dept A (60 hosts): Need $2^n - 2 \geq 60 \Rightarrow n = 6$, prefix = /26. Assign 10.0.0.0/26 (hosts .1–.62).
2. Dept B (30 hosts): Need $n = 5$, prefix = /27. Assign 10.0.0.64/27 (hosts .65–.94).
3. P2P Link 1: /30. Assign 10.0.0.96/30 (hosts .97–.98).
4. P2P Link 2: /30. Assign 10.0.0.100/30 (hosts .101–.102).
5. Remaining: 10.0.0.104/24 onward is free for future use.

Interview Corner**Q: What is VLSM?**

Variable Length Subnet Masking: Use different prefix lengths for different subnets based on actual need. A /30 for a P2P link (needs 2 hosts), a /25 for a large department (126 hosts). This wastes far fewer addresses than equal-size subnetting.

Q: What is “supernetting” (route aggregation)?

The opposite of subnetting: combining multiple contiguous smaller prefixes into one larger prefix. ISPs aggregate customer routes before advertising to reduce the size of the global BGP routing table. Example: 192.168.0.0/24, 192.168.1.0/24, 192.168.2.0/24, 192.168.3.0/24 can be aggregated into 192.168.0.0/22.

Q: Is 192.168.1.255 always a broadcast address?

No. It depends on the subnet mask. 192.168.1.255 is the broadcast for 192.168.1.0/24 only. In 192.168.0.0/23, the broadcast is 192.168.1.255 but 192.168.1.x addresses are valid hosts.

2.3 IPv6: The 128-bit Address Space

IPv4 has ~4.3 billion addresses. The Internet has over 15 billion devices (and growing). **IPv6** (RFC 8200) uses 128-bit addresses: $2^{128} \approx 3.4 \times 10^{38}$ addresses — enough for $\sim 4.8 \times 10^{28}$ addresses per person on Earth.

2.3.1 IPv6 Address Format and Notation

8 groups of 16-bit hexadecimal values separated by colons:

2001:0db8:85a3:0000:0000:8a2e:0370:7334

Abbreviation rules:

- Leading zeros in each group can be omitted: 0000 → 0.
- One contiguous sequence of all-zero groups can be replaced with :: (only once per address).
- 2001:0db8:0000:0000:0000:0000:0000:0001 → 2001:db8::1
- ::1 = loopback (all 127 bits = 0, last = 1).

2.3.2 IPv6 Address Types

Type / Prefix	Description
::1/128	Loopback (= 127.0.0.1 in IPv4).
::/0	Default route.
fe80::/10	Link-local. Auto-configured from MAC (EUI-64). Not routed. Mandatory on every interface.
fc00::/7	Unique-local (= RFC 1918 private). fd prefix most common.
2000::/3	Global unicast (public addresses, like public IPv4).
ff00::/8	Multicast (no broadcast in IPv6!). ff02::1 = all nodes, ff02::2 = all routers.
::ffff:x.x.x.x	IPv4-mapped IPv6 address (for dual-stack).
2002::/16	6to4 tunnelling (deprecated).
64:ff9b::/96	NAT64 well-known prefix (translate IPv6 → IPv4).

Deep Dive / Detail

Key IPv6 Features vs IPv4

- **No broadcast:** Uses multicast and anycast instead. Reduces network overhead.
- **No NAT needed:** Every device gets a globally unique address.
- **SLAAC (Stateless Address Autoconfiguration):** Devices self-configure using the 64-bit network prefix from Router Advertisements + EUI-64 (derived from MAC by inserting FF:FE in the middle). No DHCP server needed.
- **DHCPv6:** Stateful address assignment for more control.
- **Simplified header:** Fixed 40-byte header (IPv4 has variable 20-60B header). Extension headers added separately for options (fragmentation, routing, ESP...).
- **Flow label:** 20-bit field for identifying flows — QoS support without inspecting higher-layer headers.
- **No IP-level fragmentation by routers:** Only the source host fragments; routers use ICMPv6 “Packet Too Big” for PMTUD.
- **IPsec:** Was mandatory in original spec; now optional in practice.
- **Neighbour Discovery Protocol (NDP):** Replaces ARP. Uses ICMPv6 messages (Router Solicitation, Router Advertisement, Neighbour Solicitation, Neighbour Advertisement, Redirect).

2.3.3 IPv6 Header Structure

Version (4b)	Traffic Class (8b)	Flow Label (20b)	
Payload Length (16b)	Next Header (8b)	Hop Limit (8b)	
Source Address (128 bits)			
Destination Address (128 bits)			

Next Header field: Indicates the type of the next header (extension header or upper-layer protocol). Values: 6=TCP, 17=UDP, 43=Routing EH, 44=Fragment EH, 58=ICMPv6, 59=No next header.

2.4 ARP: Address Resolution Protocol

IP routing delivers a packet to the right *network*. But to send a frame on that network, the sender needs the **MAC address** of the next-hop device. **ARP** (RFC 826) resolves IPv4 addresses to MAC addresses.

Concept / Pattern

ARP Request/Reply Process

1. Host A wants to send to 192.168.1.5. Checks its ARP cache first.
2. Cache miss $\Rightarrow$ broadcasts ARP Request: “Who has 192.168.1.5? Tell 192.168.1.1” (dst MAC = FF:FF:FF:FF:FF:FF, ARP EtherType = 0x0806).
3. *Every* host on the subnet receives this broadcast.
4. Host B (owning .1.5) sends ARP Reply (unicast): “192.168.1.5 is at AA:BB:CC:DD:EE:FF”.
5. Host A caches the *IP* $\rightarrow$ *MAC* mapping (view with `arp -n` or `ip neigh`).
6. Host A sends the Ethernet frame with destination MAC = AA:BB:CC:DD:EE:FF.

ARP cache timeout: Typically 20 minutes on Linux (`net.ipv4.neigh.default.gc_stale_time`).

Proxy ARP: A router answers ARP requests on behalf of hosts on the other side. Enables transparent routing.

Gratuitous ARP: A host broadcasts an ARP reply for its own IP (sender IP = target IP = own IP). Used to: (1) announce a new IP, (2) update ARP caches after failover (HA/VRRP), (3) detect IP conflicts.

Common Mistake / Gotcha**ARP Spoofing (ARP Poisoning)**

ARP has *no authentication*. An attacker sends a gratuitous ARP reply claiming “10.0.0.1 is at MY-MAC”, poisoning all ARP caches on the LAN. All traffic destined for the gateway now flows through the attacker (MITM).

Defences: Dynamic ARP Inspection (DAI) on managed switches — validates ARP packets against a trusted DHCP snooping binding table (IP/MAC/port/VLAN). Private VLANs, static ARP entries, 802.1X port authentication.

2.5 NAT: Network Address Translation

NAT allows multiple devices with private (RFC 1918) IPs to share one public IP. Invented as a short-term fix for IPv4 exhaustion; still nearly universal due to slow IPv6 adoption.

Concept / Pattern**NAPT / PAT (Port Address Translation) — The Most Common NAT**

1. Client 192.168.1.5:54321 sends a packet to 8.8.8.8:53.
2. NAT router **rewrites** the source IP:port to 203.0.113.1:10001 and records the mapping in its *NAT translation table*.
3. The server (8.8.8.8) sees the request from 203.0.113.1:10001, replies to that address.
4. NAT router looks up port 10001, rewrites destination to 192.168.1.5:54321, and forwards to the internal client.

Incoming connections (port forwarding): Since external hosts can’t initiate connections through NAT (there’s no existing mapping), you must configure a static mapping: e.g., external port 8080 → internal 192.168.1.5:80.

Deep Dive / Detail**NAT Types (Critical for VoIP, Gaming, and P2P)**

- **Full-Cone (1:1) NAT:** Once a mapping (internal:port → external:port) is created, *any* external host can send to the external port. Most permissive; best for P2P.
- **Address-Restricted Cone NAT:** Only packets from a specific external IP can send back (regardless of port). Only if the internal host sent to that IP first.
- **Port-Restricted Cone NAT:** Only the specific external IP:port pair that the internal host contacted can send back.
- **Symmetric NAT:** A different external port is created for each distinct destination IP:port. Most restrictive. Breaks most NAT traversal (WebRTC TURN required).

CGNAT (Carrier-Grade NAT): ISPs use 100.64.0.0/10 (RFC 6598) as an intermediate private range, NATing many customers behind one public IP. Breaks incoming connections, makes IP-based bans/rate-limiting ineffective, causes double-NAT issues.

STUN / TURN / ICE: Used in WebRTC/VoIP to traverse NAT:

- **STUN:** Server tells you your public IP:port. Works for cone NATs.
- **TURN:** Relay server forwards all traffic. Always works but adds latency.
- **ICE:** Framework that tries STUN first, falls back to TURN.

Chapter 3

TCP: The Reliable Transport Protocol

3.1 TCP Segment Header — Every Field Explained

Deep Dive / Detail

Field	Purpose and Details
Source Port (16b)	Sending process's port (1–65535). Ephemeral ports for clients: typically 49152–65535.
Destination Port (16b)	Well-known/registered port of the destination service.
Sequence Number (32b)	Byte offset of the first byte of data in this segment. Wraps around (modulo 2^{32}).
Acknowledgment Number (32b)	Next byte the sender expects from the other side. Valid only when ACK flag is set. Cumulative.
Data Offset (4b)	TCP header length in 32-bit words. Min=5 (20B), Max=15 (60B). The rest is options.
Reserved (3b)	Must be zero.
Control Bits (9b)	NS, CWR, ECE, URG, ACK , PSH, RST , SYN , FIN .
Window Size (16b)	Receiver's available buffer space (rwnd). Max 65535B; use Window Scale option to go larger.
Checksum (16b)	Covers TCP header + data + a 12-byte pseudo-header (src IP, dst IP, protocol, TCP length).
Urgent Pointer (16b)	Byte offset from seq number to the end of urgent data. Valid only if URG flag set.
Options (0–40B)	MSS, Window Scale, SACK Permitted, SACK, Timestamps, TCP Fast Open. Padded to 32-bit boundary.

3.1.1 TCP Control Flags — What Each Does

- **SYN**: Synchronise sequence numbers. Used only during the three-way handshake.
- **ACK**: The Acknowledgment Number field is valid. Set in all packets after the initial SYN.
- **FIN**: Sender has finished sending data. Initiates graceful connection teardown.
- **RST**: Reset the connection. Immediate abort. No graceful teardown. Sent when receiving packets for a non-existent or invalid connection.
- **PSH**: Push — deliver data to the application immediately without buffering (bypass Nagle).
- **URG**: Urgent data present. Urgent Pointer field is valid.
- **ECE**: ECN Echo. Indicates the other end received a CE (Congestion Experienced) mark.
- **CWR**: Congestion Window Reduced. Acknowledges receipt of ECE.

3.2 TCP Connection Lifecycle

3.2.1 Three-Way Handshake: Establishing a Connection

Concept / Pattern

SYN → SYN-ACK → ACK — Why Three Steps?

Client	Server
-- SYN (seq=x) ----->	Client: "I want to connect; my ISN is x"
	Server allocates TCB, picks ISN y
<- SYN-ACK (seq=y, ack=x+1) --	Server: "OK; my ISN is y; I got your x"
-- ACK (ack=y+1) ----->	Client: "Got your y; we're ESTABLISHED"
[DATA TRANSFER]	

Why 3 ways? Two messages cannot prove bidirectional connectivity.

With 3: Client proves it can send (SYN), Server proves it received and can send (SYN-ACK), Client proves it received the Server's SYN (ACK of y+1).

ISN (Initial Sequence Number): Chosen pseudo-randomly (based on clock + hash of 4-tuple in modern stacks). Prevents TCP sequence prediction attacks and confusion with segments from old connections.

Half-open connections (SYN flood): An attacker sends many SYNs. Server allocates a TCB for each. The SYN queue fills. Mitigated with **SYN Cookies**.

3.2.2 Four-Way Teardown: Closing a Connection

TCP is **full-duplex**: each direction closes independently.

Concept / Pattern

FIN → ACK → FIN → ACK

Active Closer (A)	Passive Closer (B)
-- FIN (seq=u) ----->	A: "I'm done sending" (A's half closed)
<- ACK (ack=u+1) -----	B: "Got it" (B may still send data)
[half-close: A waits]	
<- FIN (seq=v) -----	B: "I'm done too"
-- ACK (ack=v+1) ----->	A: "Got it" (A enters TIME_WAIT)
[TIME_WAIT: 2*MSL = 1-4min]	
[Connection fully closed after TIME_WAIT]	

Simultaneous close: Both sides send FIN at the same time. Both enter FIN_WAIT_1 → CLOSING → TIME_WAIT.

RST instead of FIN: SO_LINGER with linger=0 causes the kernel to send RST on close(), immediately discarding buffered data. Used when you want to abort without waiting.

Deep Dive / Detail**TIME_WAIT State: Why It Exists and Why It Matters in Production**

Duration: $2 \times \text{MSL}$ (Maximum Segment Lifetime). $\text{MSL} = 60\text{s}$ (RFC 793) or 30s (Linux). So `TIME_WAIT` lasts 60–120 seconds.

Why it exists:

1. The final ACK could be lost. B would retransmit its FIN. `TIME_WAIT` lets A retransmit the ACK.
2. Prevents a new connection on the same 4-tuple from receiving delayed packets from the old connection (“confused” connection).

Production problem: A reverse proxy making thousands of outgoing connections (e.g., nginx, HAProxy) accumulates `TIME_WAIT` sockets and can exhaust ephemeral ports ($49152\text{--}65535 = 16,383$ ports).

Solutions:

- `SO_REUSEADDR`: Allow binding to port in `TIME_WAIT` for server restart.
- `net.ipv4.tcp_tw_reuse=1`: Reuse `TIME_WAIT` sockets for new outgoing connections when safe (when timestamps are available).
- `ip_local_port_range`: Widen ephemeral port range.
- Use multiple source IPs (each gets full port range).
- Use HTTP keep-alive / connection pooling (fewer connections = fewer `TIME_WAIT`s).

3.2.3 TCP State Machine

TCP has 11 defined states (RFC 793):

State	Description
LISTEN	Server waiting for incoming connections (<code>listen()</code> called).
SYN_SENT	Client sent SYN, waiting for SYN-ACK.
SYN_RCVD	Server received SYN, sent SYN-ACK, waiting for ACK.
ESTABLISHED	Connection active. Full-duplex data transfer in both directions.
FIN_WAIT_1	Active closer sent FIN, waiting for ACK or simultaneous FIN.
FIN_WAIT_2	Active closer received ACK for its FIN. Waiting for remote FIN.
CLOSE_WAIT	Passive closer received FIN, sent ACK. App has not called <code>close()</code> yet.
LAST_ACK	Passive closer sent its FIN. Waiting for the final ACK.
TIME_WAIT	Active closer received remote FIN, sent final ACK. Waiting $2 \times \text{MSL}$.
CLOSING	Both sides sent FIN simultaneously. Waiting for remote ACK.
CLOSED	No connection. Initial and final state.

Common Mistake / Gotcha**CLOSE_WAIT accumulation = application bug.**

Many `CLOSE_WAIT` sockets indicate the remote peer closed the connection but your application hasn’t called `close()` yet. The fix is always in the application: detect EOF (`read()` returning 0), handle it, and close the socket. Common in buggy thread-per-connection servers. Diagnose: `ss -tan state close-wait`.

3.3 TCP Reliability Mechanisms

3.3.1 Sequence Numbers, ACKs, and Byte Streams

TCP treats data as a **byte stream**, not messages. Every byte has a sequence number.

- **Segment seq number:** The sequence number of the *first byte* of data in this segment.
- **ACK number:** “I’ve received all bytes up to $N - 1$; I’m expecting byte N .” This is **cumulative acknowledgement**.
- **Delayed ACK:** TCP waits up to 500ms (usually 40–200ms) before sending an ACK, hoping to piggyback it on data going the other way. Disable with `TCP_QUICKACK`.
- **Retransmission Timeout (RTO):** If the sender doesn’t receive an ACK within the RTO, the segment is retransmitted. RTO starts at 1s and doubles on each retransmission (exponential backoff).

3.3.2 RTO Calculation (RFC 6298 — Jacobson/Karels Algorithm)

$$\text{SRTT} \leftarrow (1 - \alpha) \text{SRTT} + \alpha R \quad (\alpha = 1/8) \quad (3.1)$$

$$\text{RTTVAR} \leftarrow (1 - \beta) \text{RTTVAR} + \beta |\text{SRTT} - R| \quad (\beta = 1/4) \quad (3.2)$$

$$\text{RTO} = \max(\text{SRTT} + 4 \text{RTTVAR}, 1\text{s}) \quad (3.3)$$

where R is the latest measured RTT sample.

Karn’s Algorithm: Never use RTT samples from retransmitted segments (ambiguity: is the ACK for the original or the retransmission?). Use timestamps option (RFC 7323) to disambiguate.

Exponential backoff: On each consecutive timeout, double the RTO (up to 64 seconds).

3.3.3 Fast Retransmit

Waiting for the full RTO is slow (often 200ms–1s+). If the receiver gets out-of-order segments, it sends **duplicate ACKs** (same ACK number) for each subsequent out-of-order segment. **Three duplicate ACKs** signals likely loss: the sender *immediately retransmits* the missing segment without waiting for RTO.

3.3.4 Selective Acknowledgements (SACK)

Standard TCP must retransmit all segments after the first loss (go-back-N). **SACK** (RFC 2018) allows the receiver to report exactly which byte ranges have been received, so the sender only retransmits the *specific missing segments*.

Example / Code

Segments 1, 2, 4, 5, 6 received; segment 3 lost. SACK blocks:
 ACK=3, SACK=[4–6] — “Give me segment 3; I already have 4 through 6.”
 Sender only retransmits segment 3.

SACK negotiation: Both sides must announce `SACK-Permitted` in the SYN/SYN-ACK.

3.3.5 D-SACK (Duplicate SACK, RFC 2883)

Extends SACK to report *duplicate* data that was received. Allows the sender to infer:

- The ACK was lost (not the data) — the sender retransmitted unnecessarily.
- Packets are being reordered in the network.
- Allows smarter retransmission decisions and faster recovery.

3.4 TCP Flow Control: Sliding Window

Flow control prevents the sender from overwhelming the receiver’s buffer.

Concept / Pattern**Receive Window (rwnd)**

The receiver advertises how many bytes of buffer space it has available in *every ACK*. The sender must not have more than **rwnd** unacknowledged bytes in flight:

$$\text{bytes in flight} \leq \text{rwnd}$$

- As the application reads data from the receive buffer, **rwnd** increases and the sender can transmit more.
- **Zero window**: Receiver's buffer is full. Sender must stop sending.
- **Zero Window Probe**: Sender periodically sends a 1-byte segment to elicit a window update, preventing deadlock (sender waiting for window update, receiver waiting for data to read).
- **Window Scale Option** (RFC 7323): The 16-bit window field caps at 65535B. For high-bandwidth-delay-product (BDP) links, this is far too small. Window Scale multiplies the window by 2^k (up to $k = 14$, giving max 2^{30} bytes = 1 GiB). Must be negotiated in the handshake.

BDP (Bandwidth-Delay Product): $\text{BDP} = \text{bandwidth} \times \text{RTT}$. For a 1 Gbps link with 100ms RTT: $\text{BDP} = 10^9 \times 0.1 = 10^8$ bytes = 100 MB. Without Window Scaling, TCP is limited to 65535B in flight, achieving at most $65535/0.1 = 655$ Kbps on that link!

Nagle's Algorithm (RFC 896): Buffer small sends. Send data immediately only if:

1. The buffered data fills a full MSS, OR
2. All previously sent data has been ACK'd (the "pipe is clear").

Otherwise, buffer the data. Prevents the "tinygram problem" where every `write()` call sends a tiny TCP segment with just 1–2 bytes (telnet typing SSH keystrokes).

Disable with TCP_NODELAY for latency-sensitive apps: SSH, Redis, gaming.

TCP_CORK (Linux): Accumulate data until a full MSS or until uncorked. Useful for file sends.

3.5 TCP Congestion Control

Flow control manages the *receiver's* capacity. Congestion control manages the *network's* capacity. The sender maintains a **congestion window (cwnd)**.

$$\text{effective send rate} = \frac{\min(\text{rwnd}, \text{cwnd})}{\text{RTT}}$$

3.5.1 TCP Reno / Tahoe Phases

Deep Dive / Detail

Phase 1 — Slow Start (from connection start or after timeout/idle):

- Start: $\text{cwnd} = 1 \text{ MSS}$ (or 10 MSS in modern implementations, RFC 6928).
- Each ACK: $\text{cwnd} += 1 \text{ MSS}$. This doubles cwnd *every* RTT (exponential growth).
- Stop when $\text{cwnd} \geq \text{ssthresh}$ (slow start threshold) → enter Congestion Avoidance.

Phase 2 — Congestion Avoidance (probing):

- Each ACK: $\text{cwnd} += \text{MSS}^2 / \text{cwnd} \approx 1 \text{ MSS per RTT}$ (linear growth).
- Probes for available bandwidth slowly.

Loss via Timeout (severe congestion):

- $\text{ssthresh} = \text{cwnd}/2$, $\text{cwnd} = 1 \text{ MSS}$. Restart Slow Start.
- Tahoe always goes back to 1 MSS. Reno adds Fast Recovery.

Loss via 3 Duplicate ACKs (mild congestion) — Reno Fast Recovery:

- $\text{ssthresh} = \text{cwnd}/2$, $\text{cwnd} = \text{ssthresh} + 3 \text{ MSS}$.
- Retransmit the lost segment. For each additional dup ACK: $\text{cwnd} += 1 \text{ MSS}$ (inflate).
- When a new (non-dup) ACK arrives: $\text{cwnd} = \text{ssthresh}$. Return to Congestion Avoidance.

3.5.2 TCP CUBIC (Linux Default Since 2.6.19)

Instead of AIMD's linear growth, CUBIC uses a **cubic function** of time since the last congestion event:

$$W(t) = C \cdot (t - K)^3 + W_{\max}$$

where $W_{\max}$ is the window when the last loss occurred, K is the time to reach $W_{\max}$, and C is a scaling constant ($C = 0.4$).

CUBIC probes aggressively initially, slows near $W_{\max}$ (where congestion happened before), then probes again.

Advantage: Much better utilisation on high-BDP links (10 Gbps transoceanic) where Reno's linear growth is too slow to fill the pipe.

3.5.3 BBR (Bottleneck Bandwidth and Round-trip propagation time)

Developed by Google (2016, RFC in progress). **Key insight:** Traditional loss-based congestion control mistakes *buffer bloat* (latency increase from full router buffers) for congestion. BBR models two physical quantities directly:

- **BtlBw** (bottleneck bandwidth): Measured by tracking the maximum delivery rate.
- **RTprop** (round-trip propagation time): Measured as the minimum RTT.

$$\text{cwnd} = \text{BtlBw} \times \text{RTprop} \times \text{gain}$$

BBR periodically enters “ProbeRTT” (reduces inflight to 4 MSS for 200ms) to measure the true propagation delay without queuing. And “ProbeBW” to probe for more bandwidth.

Advantages: Better throughput on lossy networks (Wi-Fi, satellite), lower latency (avoids filling buffers). Used by Google globally. Some controversy: unfair to CUBIC in shared buffers.

Interview Corner**Q: Why is Slow Start “slow”?**

It starts at `cwnd=1 MSS` (or 10 MSS) rather than full line rate. The name is historical — before TCP’s congestion control, senders would blast at full speed and overwhelm routers. Despite starting slow, it doubles every RTT: after 10 RTTs at 50ms each (500ms), $\text{cwnd} = 2^{10} \approx 1024 \text{ MSS} \approx 1.4 \text{ MB}$. Most short connections (Google search) are still in Slow Start when they finish.

Q: What is the difference between `cwnd` and `rwnd`?

`cwnd` (congestion window): Maintained by the *sender*, reflects the network’s capacity. Dynamic. `rwnd` (receive window): Advertised by the *receiver*, reflects available buffer space. The sender uses $\min(\text{cwnd}, \text{rwnd})$.

Q: Explain why 3 duplicate ACKs indicates packet loss.

When segment N is lost, segments $N + 1, N + 2, \dots$ arrive out of order at the receiver. The receiver sends a duplicate ACK ($\text{ACK} = N$) for each subsequent segment. By the time 3 dup ACKs arrive, 3 segments have arrived after the hole. It’s unlikely (but possible) that 3 segments arrived out of order due to reordering alone, so 3 is the heuristic threshold. *Note: RACK (RFC 8985) is a newer algorithm using timestamps, less sensitive to reordering.*

3.6 TCP Options Reference

Option			Purpose
MSS	(Kind=2, 4B)		Max Segment Size. Announced by each side in SYN/SYN-ACK. Typical: 1460B (Ethernet MTU 1500 - 40B headers).
Window	Scale	(Kind=3, 3B)	Multiply the 16-bit window field by 2^k , $k \in [0, 14]$. Negotiated in handshake.
SACK-Permitted		(Kind=4)	“I support Selective ACKs.” Negotiated in handshake.
SACK	(Kind=5, variable)		Up to 4 SACK blocks per segment. Each block = [left edge, right edge] of received bytes.
Timestamps	(Kind=8, 10B)		Two 32-bit timestamps. Used for RTT measurement (TSval sent by sender, TSecr echoed). Also for PAWS.
PAWS			Protection Against Wrapped Sequence Numbers: reject old segments from previous connections using timestamps.
TCP	Fast	Open	(Kind=34) TFO. Allows sending data in the SYN packet on repeat connections. Saves 1 RTT. Server stores a cookie per client.

Chapter 4

UDP and Lightweight Transport Protocols

4.1 UDP Header — Minimal Design

UDP (User Datagram Protocol, RFC 768) has an 8-byte header:

Source Port (16b)	Destination Port (16b)	Length (16b)	Checksum (16b)
-------------------	------------------------	--------------	----------------

- **Length:** Total length of UDP header + data in bytes. Minimum = 8 (header only).
- **Checksum:** Optional in IPv4 (set to 0 to disable), mandatory in IPv6. Covers UDP header + data + IPv4 pseudo-header.
- No connection setup, no ACK, no ordering, no retransmission, no congestion control.

4.2 When UDP is the Right Choice

Concept / Pattern

Use UDP When:

- **Latency matters more than reliability:** VoIP (Skype, WhatsApp), video calls (Zoom, Meet), live streaming. A late packet is *worse* than a missing one (the speaker has already moved on; playing the old audio would sound garbled).
- **Simple request-response:** DNS (query + response = 2 packets; TCP handshake overhead is unacceptable), DHCP, SNMP, TFTP. The application retries on its own.
- **Broadcast/Multicast:** UDP supports `FF:FF:FF:FF:FF:FF` broadcasts and multicast groups. TCP is strictly unicast.
- **Application-layer reliability:** QUIC, WebRTC (DTLS/SRTP over UDP), game engines with custom reliability. They get full control over what to retransmit.
- **Real-time media:** RTP (Real-time Transport Protocol, RFC 3550) over UDP. Late packets are discarded; FEC (Forward Error Correction) handles some losses.

Deep Dive / Detail

RTP (Real-time Transport Protocol) — Used in All VoIP and Video

- Runs over UDP. Adds timestamp (for playout synchronisation), sequence number (detect loss/re-order), SSRC (stream identifier), payload type (codec: G.711, Opus, VP8...).
- **RTCP** (RTP Control Protocol): Companion protocol for statistics and control. Reports: packet loss rate, jitter, RTT. Both peers can adapt codec/bitrate.
- **Jitter buffer**: The receiver buffers 50–200ms of RTP packets, smoothing out network jitter (variable delay). Trades latency for smoothness.
- **SRTP** (Secure RTP): Adds encryption (AES-CTR), authentication (HMAC-SHA1) to RTP. Used in WebRTC.

4.3 QUIC: The Future of Transport

QUIC (Quick UDP Internet Connections, RFC 9000, 2021) is a new transport protocol built on UDP, developed by Google. HTTP/3 (RFC 9114) uses QUIC as its transport.

Deep Dive / Detail

Why QUIC? The Problems It Solves**Problem 1 — TCP Head-of-Line Blocking:**

HTTP/2 multiplexes many logical streams over one TCP connection. TCP delivers bytes in strict order. If one segment is lost, *all streams* stall, even if other streams' data arrived safely.

QUIC fix: Loss of a QUIC packet blocks only the specific stream whose data it carried.

Problem 2 — Connection Establishment Latency:

TCP (1 RTT) + TLS 1.3 (1 RTT) = 2 RTTs before first byte of HTTP data.

QUIC fix: Combines transport and crypto handshake = **1 RTT** for new connections.

For resumed connections: **0-RTT** — client sends data with the very first packet.

Problem 3 — Connection Migration:

TCP connections are tied to the client's IP:port 4-tuple. Moving from Wi-Fi to mobile data changes your IP, killing all TCP connections.

QUIC fix: Uses a **Connection ID** (64-bit, negotiated in handshake). The connection migrates seamlessly when the network interface changes.

Problem 4 — Ossification:

NAT boxes and middleboxes “know” TCP and may corrupt options or block certain features. QUIC runs over UDP (already widely unblocked) and is fully encrypted — middleboxes can't see or modify internals.

0-RTT Security Note: 0-RTT data is not replay-protected. The server must make 0-RTT idempotent (safe to replay). POST requests should not be sent as 0-RTT.

Feature	TCP + TLS 1.3	QUIC
New connection RTTs	2 RTT	1 RTT
Resumed connection	1–2 RTT	0 RTT
HoL blocking	Full HOL blocking	Per-stream (no cross-stream HOL)
Connection migration	Not supported	Supported (Connection ID)
Encryption	TLS on top of TCP	Integrated TLS 1.3
Middlebox transparency	Partial	Fully encrypted
OS kernel involvement	TCP in kernel	UDP in kernel; QUIC in userspace

4.4 SCTP: Stream Control Transmission Protocol

SCTP (RFC 4960) is a third transport option:

- **Multi-streaming:** Multiple independent streams within one association. Loss in one stream doesn't block others.
- **Multi-homing:** An association can use multiple IP addresses. If one link fails, traffic switches to another.
- **Message-oriented:** Preserves message boundaries (unlike TCP's byte stream).
- **4-way handshake:** Uses a cookie mechanism to prevent SYN floods.
- **Used in:** Telecom (SS7 over IP), LTE control plane (S1AP), WebRTC signalling.

Interview Corner

Q: Why doesn't QUIC replace TCP entirely?

QUIC is a userspace protocol. Implementing it in userspace means more CPU overhead (vs. kernel TCP) and slower adoption (every app must use a QUIC library). Kernel TCP benefits from decades of tuning. Also, many middleboxes block UDP or deprioritise it vs TCP. QUIC adoption is growing rapidly (30%+ of Google traffic, 28% of all web traffic) but TCP will remain dominant for years.

Q: UDP has no congestion control. Doesn't it just ruin the network?

Unrestricted UDP flooding is a real problem (video streaming apps historically hammered networks). QUIC implements its own congestion control (BBR or Cubic). Well-designed UDP apps include congestion control. But "UDP floods" for gaming/streaming without CC are a real concern for network fairness, addressed by traffic shaping at ISPs.

Chapter 5

DNS: The Internet's Directory Service

5.1 DNS Architecture and Hierarchy

DNS (Domain Name System, RFC 1034/1035, 1987) is a globally distributed, hierarchical, delegated database mapping names to data (primarily hostnames → IP addresses).

Concept / Pattern

The DNS Hierarchy (Tree Structure)



- **Root name servers:** 13 logical servers (A–M.root-servers.net). Hundreds of physical servers via anycast. Operated by 12 organisations. Know the IP of every TLD authoritative server.
- **TLD servers:** .com, .org, .in, .uk. Know the authoritative NS servers for each registered domain.
- **Authoritative name servers:** Hold the actual DNS records for a domain. ns1.google.com knows www.google.com → 142.250.80.36.
- **Recursive resolver (caching resolver):** Your ISP's DNS server, Google's 8.8.8.8, Cloudflare's 1.1.1.1. Does the full recursive lookup on your behalf and caches results (respecting TTLs).

5.2 Full DNS Resolution Walk-Through

When you type `www.example.com` in your browser:

1. **Browser cache:** Chrome caches DNS for up to 1 minute regardless of TTL. `chrome://net-internals/#dns`
2. **OS cache + /etc/hosts:** /etc/hosts checked first (always). Then OS DNS cache.
3. **Stub resolver** queries the configured **recursive resolver** (e.g., 8.8.8.8) via UDP port 53.
4. **Resolver cache check:** If it cached `www.example.com`'s A record, returns immediately.
5. **Resolver queries a Root server:** "What are the NS servers for .com?" Root says: `a.gtld-servers.net`.
6. **Resolver queries the .com TLD server:** "What are the NS servers for example.com?" TLD replies with NS records for example.com (e.g., `ns1.example.com`, `ns2.example.com`).
7. **Resolver queries example.com's authoritative NS:** "What is the A record for www.example.com?" NS replies: A 93.184.216.34 TTL=3600.
8. Resolver caches the A record for TTL=3600 seconds. Returns IP to OS.

9. OS caches, browser caches, application gets the IP.
10. Browser opens TCP/TLS connection to 93.184.216.34:443.

DNS uses UDP port 53 for queries (responses ≤ 512 B historically, extended to 4096B with EDNS0). Falls back to TCP port 53 if the response exceeds the EDNS0 limit. Zone transfers (**AXFR/IXFR**) always use TCP.

5.3 DNS Record Types — Complete Reference

Type	Name	Value / Purpose
A	Address (IPv4)	<code>example.com. IN A 93.184.216.34.</code> Maps host-name to IPv4.
AAAA	IPv6 Address	Maps hostname to IPv6.
CNAME	Canonical Name	Alias: <code>www.example.com. CNAME example.com..</code> Client resolves CNAME target.
MX	Mail Exchange	<code>10 mail.example.com..</code> Priority (lower = preferred) + mail server hostname.
NS	Name Server	<code>example.com. NS ns1.example.com..</code> Delegates authority to the NS.
TXT	Text	SPF, DKIM public keys, domain ownership proofs (Google/AWS), DMARC. Free-form text.
PTR	Pointer (reverse)	Maps IP to hostname. <code>34.216.184.93.in-addr.arpa. PTR www.example.com..</code> Used to verify IPs.
SOA	Start of Authority	Zone metadata: primary NS, admin email (. instead of @), serial number, refresh, retry, expire, minimum TTL.
SRV	Service Location	<code>_xmpp._tcp.example.com. SRV 10 0 5222 xmpp.example.com.</code> Priority, weight, port, target. Used by SIP, XMPP, etc.
CAA	Cert Authority	Which CAs are allowed to issue TLS certs for this domain. E.g., <code>0 issue "letsencrypt.org".</code>
DNSKEY	DNSSEC Key	Public key for validating DNSSEC signatures.
DS	Delegation Signer	Hash of a child zone's DNSKEY. Establishes chain of trust for DNSSEC.
RRSIG	Resource Record Sig	DNSSEC signature covering a set of resource records.
NAPTR	Naming Auth Ptr	Used in VoIP (SIP/ENUM) for telephone number to URI mapping.
TLSA	TLS Auth	DANE: Pin the TLS cert/key for a domain to its DNS record.

Common Mistake / Gotcha

CNAME Restrictions

CNAME cannot coexist with *any* other record at the same name. You cannot have a CNAME at the zone apex (`example.com.`) because the zone also needs SOA and NS records. You also cannot have a CNAME at the same name as an MX record.

Solutions: ALIAS/ANAME records (non-standard but supported by Cloudflare, Route53, NS1) which behave like CNAME but are resolved at the authoritative NS. Or use multiple A records.

5.4 DNS Caching, TTL, and Negative Caching

- **TTL (Time-To-Live):** Each record has a TTL in seconds. Resolvers cache for the TTL. TTL=300 (5 min) = change propagates worldwide in 5 min. TTL=86400 (24h) = up to 24h for propagation.
- **Pre-migration TTL reduction:** Lower TTL to 60s *several days before* a planned IP change (wait for all old high-TTL caches to expire). Do the cutover. Restore TTL to 3600–86400 once verified.

- **Negative caching (RFC 2308):** NXDOMAIN (name doesn't exist) responses are also cached, for the SOA record's minimum TTL. Prevents repeated queries for non-existent names (typos).
- **Stale-while-revalidate:** Some resolvers can serve stale (expired TTL) cached records while fetching fresh ones in the background. Reduces latency at the cost of freshness.

5.5 DHCP: Dynamic Host Configuration Protocol

DHCP (RFC 2131) automatically configures hosts: IP address, subnet mask, default gateway, DNS servers, NTP servers, domain name.

Concept / Pattern

DHCP DORA Process

Client	Server
-- DHCPDISCOVER (broadcast) ----->	"Anyone there? I need an IP"
<-- DHCPOFFER (broadcast/unicast) --	"Here's 192.168.1.50/24, GW .1, DNS .1"
-- DHCPREQUEST (broadcast) ----->	"I'll take that IP offer"
<-- DHCPACK (unicast) -----	"Confirmed. Lease is 24 hours."
[Client configures interface]	

Discover → Offer → Request → Acknowledge.

Why broadcast?: Client doesn't have an IP yet, so it sends from 0.0.0.0 to 255.255.255.255. Only the DHCP server processes it (using UDP ports 67/68).

Lease renewal: Client tries to renew at T/2 (50% of lease time) by unicasting DHCPREQUEST to the original server. If no response, broadcasts at T*0.875. On lease expiry, client must stop using the IP.

DHCP Relay Agent: Routers forward DHCP broadcasts across subnet boundaries to a centralised DHCP server.

Deep Dive / Detail

DHCPv6 vs SLAAC

SLAAC (Stateless Address Autoconfiguration, RFC 4862): Host generates its own IPv6 address using the network prefix from Router Advertisements (RA) + interface identifier (EUI-64 from MAC, or random Privacy Extensions RFC 8981). No server needed. No record of which host has which IP. Limited configuration (just IP + prefix + gateway from RA; no DNS via RA alone).

DHCPv6 (Stateful): Like DHCPv4 — server assigns addresses and tracks leases. Full control. Provides DNS servers (Option 23), domain search list.

RDNSS (RFC 8106): Extends RA to include DNS server addresses. Allows full SLAAC without DHCPv6. Now widely supported.

In practice: Most networks use SLAAC + RDNSS for clients, DHCPv6 for servers needing stable predictable addresses.

5.6 DNS Security: Attacks and Defences

Deep Dive / Detail

Attack 1: DNS Cache Poisoning (Kaminsky Attack, 2008)

DNS uses a 16-bit transaction ID (0–65535). Attacker races to send many forged UDP “responses” with all possible TXIDs before the real authoritative server responds. If a guess is correct, the resolver caches the attacker’s fake record (e.g., `bank.com` → `attacker-IP`).

Fix: Source port randomisation (also randomise the source port: $65535 \times 65535 \approx 4$ billion combinations). **Full fix:** DNSSEC.

Attack 2: DNS Amplification DDoS

Attacker sends small DNS ANY queries (60B) with the victim’s IP as the source. Authoritative servers respond with large replies (3000B+) to the victim. Amplification ratio: 50x. Reflection: attacker is hidden. **Mitigation:** Rate limit DNS responses per source IP, BCP38 (ISPs filter packets with spoofed source IPs).

DNSSEC (DNS Security Extensions, RFC 4033/4034/4035):

Cryptographically signs DNS records. Each zone has a public-private key pair. Records are signed with the private key (RRSIG records). Resolvers verify with the public key (DNSKEY records). Chain of trust: `root` → `TLD` → `domain zone`, each level signing the next’s DNSKEY hash (DS record). *DNSSEC does not encrypt* — it protects integrity and authenticity only. Adds overhead; ~30% of DNS queries are validated.

DoH (DNS over HTTPS): DNS queries sent as HTTPS to port 443 (`/dns-query` endpoint). Encrypts queries from ISP observation. Browsers use a DoH server directly (Cloudflare 1.1.1.1, Google 8.8.8.8), bypassing the local ISP resolver. Controversial: bypasses corporate DNS firewalls, centralists DNS control.

DoT (DNS over TLS, RFC 7858): DNS over a TLS connection to port 853. Same goal as DoH but uses a distinct port (easier for admins to monitor/block/allow).

ECH (Encrypted Client Hello): Extends TLS to hide the SNI (server name) from network observers. Works with DoH to hide the domain being visited.

Interview Corner

Q: What is the difference between authoritative DNS and recursive DNS?

Authoritative DNS: Holds the actual records for a domain (A, MX, CNAME...). Only answers queries about its own zones. Run by domain owners or DNS hosting providers (Cloudflare DNS, Route53, GoDaddy).

Recursive DNS: Queries other servers on your behalf, following the chain from `root` → `TLD` → `authoritative`. Caches results. Run by ISPs, Google (8.8.8.8), Cloudflare (1.1.1.1). Also called a “caching resolver” or “stub resolver’s upstream”.

Q: What happens if a DNS TTL is 0?

The record must be fetched fresh on every query — no caching. Useful for extremely dynamic IP assignments (anycast, canary deployments) but generates massive load on authoritative servers. Many resolvers enforce a minimum TTL of 30-60s anyway. Not recommended in practice.

Chapter 6

HTTP, HTTPS, and the Application Protocol Stack

6.1 HTTP Evolution: From 0.9 to 3

Deep Dive / Detail	
Version	Key Characteristics
HTTP/0.9	One-line protocol (GET /path). Only GET. TCP connection per request. No headers, no status codes.
HTTP/1.0	Headers added. Status codes. Content-Type . Multiple methods. Still one TCP connection per request (connection: close).
HTTP/1.1	Persistent connections (keep-alive by default). Pipelining (send multiple requests without waiting; broken in practice due to HoL). Host header (enables virtual hosting). Chunked transfer, range requests, conditional requests (ETag/If-None-Match).
HTTP/2	Binary framing (not text). Multiplexing : many streams on one TCP connection. HPACK header compression (dictionary-based; headers sent incrementally). Server Push (deprecated in practice). Stream prioritisation. Still suffers from TCP HoL blocking.
HTTP/3	QUIC-based (UDP). Multiplexing without TCP HoL blocking. Integrated TLS 1.3. Connection migration. 0-RTT.

6.2 HTTP Request and Response Structure

Example / Code

HTTP/1.1 Request (annotated):

```

GET /api/users/123 HTTP/1.1      <- Method SP Request-URI SP Version CRLF
Host: api.example.com           <- Required in HTTP/1.1 (enables virtual hosting)
User-Agent: Mozilla/5.0 Chrome/120 <- Identifies the client software
Accept: application/json        <- What MIME types client understands
Accept-Encoding: gzip, br, deflate <- Compression algorithms accepted
Accept-Language: en-US,en;q=0.9 <- Language preference with quality values
Connection: keep-alive          <- Keep TCP connection open after response
Authorization: Bearer eyJhbGci... <- Auth token (not auto-sent; set by JS/app)
Cache-Control: no-cache         <- Don't serve from cache (must revalidate)
If-None-Match: "686897696a7c876b" <- Conditional: send 304 if ETag unchanged
Cookie: session=abc; theme=dark  <- Cookies (HttpOnly are invisible to JS)
                                <- Empty CRLF line separates headers from body
                                <- GET has no body; POST/PUT have body here

```

HTTP/1.1 Response (annotated):

```

HTTP/1.1 200 OK                 <- Version SP Status-Code SP Reason CRLF
Content-Type: application/json   <- Body MIME type; browser uses for parsing
Content-Length: 248             <- Body size in bytes (for persistent connections)
Content-Encoding: gzip          <- Body is gzip-compressed
Cache-Control: max-age=3600, public <- Cache for 1 hour; OK to cache in CDN
ETag: "686897696a7c876b"       <- Opaque version identifier for conditional
    requests
Last-Modified: Wed, 21 Oct 2024 ... <- Alternative for conditional requests
Vary: Accept-Encoding           <- Cache must store separate entries per encoding
Set-Cookie: session=xyz; HttpOnly; Secure; SameSite=Strict; Path=/; Max-Age=3600
Strict-Transport-Security: max-age=31536000; includeSubDomains; preload <- HSTS
X-Request-Id: 550e8400-e29b-41d4... <- Non-standard debug/tracing header
                                <- Empty CRLF line
{"id": 123, "name": "Alice", ...} <- Body (gzip-compressed in flight)

```

6.3 HTTP Methods

Method	Purpose	Properties
GET	Retrieve resource	Safe + Idempotent. Body discouraged. Cacheable.
HEAD	Headers only, no body	Safe + Idempotent. Used to check resource metadata, test links.
POST	Create / submit	Neither safe nor idempotent. Not cacheable by default.
PUT	Replace entire resource	Not safe. Idempotent (same result if repeated).
PATCH	Partial update	Not safe. Not necessarily idempotent (depends on implementation).
DELETE	Delete resource	Not safe. Idempotent. Returns 204 or 200.
OPTIONS	Check allowed methods	Safe + Idempotent. Used for CORS preflight .
CONNECT	Establish tunnel	Used by HTTP proxies to create SSL tunnels to HTTPS sites.
TRACE	Echo request (debug)	Safe. Usually disabled (XST cross-site tracing attack risk).

Key Insight

Safe vs Idempotent

Safe: Does not modify server state (GET, HEAD, OPTIONS, TRACE). Can be cached, prefetched, bookmarked freely.

Idempotent: Calling it N times has the same effect as calling it once (GET, HEAD, PUT, DELETE, OPTIONS). Safe for retries on network failures.

All safe methods are idempotent; not all idempotent methods are safe (DELETE is idempotent but not safe).

Why it matters: Idempotency determines whether you can safely retry a request without causing side effects. POST is not idempotent: retrying may create duplicate orders.

6.4 HTTP Status Codes — Complete Reference

Code	Name	When to Use / What It Means
1xx Informational		
100	Continue	Client should continue sending request body.
101	Switching Protocols	Used for WebSocket upgrade.
2xx Success		
200	OK	Standard success with body.
201	Created	POST/PUT created a resource. Include Location header.
202	Accepted	Request received but not yet processed (async).
204	No Content	Success, no body (DELETE, OPTIONS, webhook ACK).
206	Partial Content	Range request result (video streaming, resumable download).
3xx Redirection		
301	Moved Permanently	Permanent redirect. Browser + search engines update. SEO transfers link juice.
302	Found (Temporary)	Temporary redirect. Browser doesn't update bookmarks. SEO keeps old URL.
303	See Other	POST-redirect-GET after form submission. Prevents re-POST on refresh.
304	Not Modified	ETag/Last-Modified matched: use the cached version. No body.
307	Temporary Redirect	Like 302 but preserves HTTP method (POST stays POST).
308	Permanent Redirect	Like 301 but preserves HTTP method.
4xx Client Errors		
400	Bad Request	Malformed syntax, invalid parameters, constraint violation.
401	Unauthorized	Not authenticated. Include WWW-Authenticate header.
403	Forbidden	Authenticated but lacks permission.
404	Not Found	Resource doesn't exist at this URL.
405	Method Not Allowed	Wrong HTTP method. Include Allow header.
408	Request Timeout	Client too slow to send the full request.
409	Conflict	State conflict (duplicate resource, concurrent edit, out-of-date ETag).
410	Gone	Resource permanently removed. Stronger signal than 404 for SEO.
413	Content Too Large	Request body exceeds server limit (file upload size).
415	Unsupported Media Type	Content-Type not accepted by this endpoint.
422	Unprocessable Entity	Syntactically OK but semantically invalid (validation errors).
429	Too Many Requests	Rate limited. Include Retry-After header.
5xx Server Errors		
500	Internal Server Error	Unhandled exception or bug.
501	Not Implemented	HTTP method not supported by this server.
502	Bad Gateway	Upstream server returned invalid/unexpected response.
503	Service Unavailable	Server overloaded or down for maintenance. Include Retry-After.
504	Gateway Timeout	Upstream server timed out.
507	Insufficient Storage	Server ran out of disk space (WebDAV).

6.5 HTTP Caching

Concept / Pattern

Cache-Control Directives — Complete Reference

Response directives (server tells caches what to do):

- **max-age=N**: Cache for N seconds from the request time.
- **s-maxage=N**: Like max-age but only for shared caches (CDNs, proxies). Overrides max-age for shared caches.
- **no-cache**: Cache may store, but must revalidate with origin on every use (ETag/Last-Modified). Does NOT mean “don’t cache”.
- **no-store**: Never cache. Not in RAM, not on disk. For sensitive data (banking, medical).
- **public**: Can be cached by any cache (browser, CDN, proxy).
- **private**: Only browser cache. Not CDN/shared. For user-specific data.
- **immutable**: Resource will never change at this URL. Browser skips revalidation even on user refresh. Use with content-hashed URLs.
- **stale-while-revalidate=N**: Serve stale content immediately while fetching fresh content in the background (within N seconds of expiry).
- **stale-if-error=N**: Serve stale content if origin is unreachable (within N seconds of expiry).

ETag (Entity Tag): Server generates a hash/version ID of the resource.

1. Server: ETag: "v3abc123"
2. Client caches. On next request: If-None-Match: "v3abc123"
3. If unchanged: server replies 304 Not Modified (no body, saves bandwidth).
4. If changed: server replies 200 with new body and new ETag.

Best practice for static assets: `max-age=31536000, immutable + content-hashed URL (/app.a1b2c3d4.js)`. When content changes, URL changes, instantly busting all caches.

6.6 Cookies, Sessions, and Authentication

Concept / Pattern

Cookie Attributes Explained

Set-Cookie: session=abc; Domain=.example.com; Path=/; Max-Age=3600;
HttpOnly; Secure; SameSite=Strict

- **Domain**: Which domains receive this cookie. Leading dot (.example.com) includes subdomains.
- **Path**: URL path scope. Only sent for URLs matching this path prefix.
- **Max-Age**: Seconds until expiry (preferred over Expires which uses an absolute date).
- **HttpOnly**: JavaScript *cannot read* this cookie (document.cookie won’t show it). Mitigates XSS stealing session tokens.
- **Secure**: Only sent over HTTPS connections. Never over plain HTTP.
- **SameSite**:
 - **Strict**: Cookie never sent on cross-site requests. Breaks OAuth flows, third-party embeds.
 - **Lax** (default since 2020): Sent on top-level GET navigations (clicking a link). Blocks CSRF from images/forms. *The recommended default.*
 - **None**: Always sent. Must also set Secure. For third-party cookies (ads, OAuth).

Deep Dive / Detail**Session Management Approaches**

- 1. Server-side sessions:** Server creates session record in storage (Redis/DB), sends session ID in `HttpOnly` cookie. Each request: server looks up session. Session data on server. Easy to revoke. Requires shared session storage in multi-server setup.
- 2. JWT (JSON Web Tokens):** Stateless. Server signs a token containing user data. Client stores and sends it (Bearer auth header or cookie). Server verifies signature (no DB needed for each request). Problems: cannot revoke individual tokens before expiry; tokens can be large; storing in `localStorage` exposes to XSS.
- 3. Best practice:** Short-lived access tokens (JWTs, 15min) for API requests + long-lived refresh tokens in `HttpOnly` cookies. Refresh tokens can be revoked in a DB (block-list or allowlist). On each API call, verify JWT signature; periodically exchange refresh token for new access token.

6.7 CORS: Cross-Origin Resource Sharing

The browser's **Same-Origin Policy (SOP)** prevents JavaScript on `a.com` from reading responses from `b.com`. Origin = scheme + host + port. **CORS** is a standard way for servers to relax SOP for trusted origins.

Concept / Pattern**CORS Flow**

Simple requests (GET/HEAD/POST + simple headers like `Accept`, `Content-Type=text/plain`):

1. Browser adds `Origin: https://a.com` to the request.
2. Server responds with `Access-Control-Allow-Origin: https://a.com` (or `*`).
3. Browser allows JS to read the response.

Preflight (OPTIONS) requests (triggered by: PUT/DELETE/PATCH methods, or custom headers, or `Content-Type=application/json`):

1. Browser sends `OPTIONS` request with: `Origin: https://a.com`,
`Access-Control-Request-Method: DELETE`, `Access-Control-Request-Headers: Authorization`
2. Server responds with: `Access-Control-Allow-Origin: https://a.com`,
`Access-Control-Allow-Methods: GET, POST, DELETE`, `Access-Control-Allow-Headers: Authorization`, `Access-Control-Max-Age: 86400`
3. Browser caches the preflight result for 86400s, then proceeds with the actual request.

Common Mistake / Gotcha**Access-Control-Allow-Origin: * and Credentials**

If you set `Access-Control-Allow-Origin: *`, the browser will *not* send cookies or Authorization headers with cross-origin requests even if you ask it to. To allow credentials: you must specify the exact origin (`Access-Control-Allow-Origin: https://a.com`) AND add `Access-Control-Allow-Credentials: true`. The wildcard (`*`) is incompatible with credentials by design.

Chapter 7

TLS/SSL: Transport Layer Security

7.1 What TLS Provides

TLS (Transport Layer Security) (originally SSL, but SSL is deprecated) provides:

- **Confidentiality:** Data is encrypted. Nobody can read it in transit — not your ISP, not your government, not someone on the same Wi-Fi.
- **Authentication:** The server's certificate proves it is who it claims to be. You are really talking to `google.com`, not an impostor.
- **Integrity:** Data cannot be modified in transit without detection. Each record has an AEAD (Authenticated Encryption with Associated Data) tag.
- **Forward Secrecy:** (in TLS 1.3) Past sessions cannot be decrypted even if the server's long-term private key is later compromised.

7.2 TLS 1.3 Handshake (1-RTT)

Concept / Pattern

Full TLS 1.3 Handshake (RFC 8446)

Client	Server
-- ClientHello ----->	
TLS version, random (32B),	
cipher suites, key_share	
(ECDHE public key, e.g. X25519)	
<-- ServerHello -----	
chosen cipher, random,	
key_share (server's ECDHE pubkey)	
[Session keys derived HERE	
from both key_shares + randoms]	
<-- {EncryptedExtensions} -----	Encrypted from here
<-- {Certificate} -----	(session key derived
<-- {CertificateVerify} -----	after ServerHello)
<-- {Finished} -----	
-- {Finished} ----->	
<==== Application Data (HTTP) =====>	1-RTT total

0-RTT for resumed connections: Client sends data with the very first packet using a PSK (Pre-Shared Key) from a previous session. Not replay-safe; use only for idempotent requests.

7.2.1 Key Exchange: ECDHE (Elliptic Curve Diffie-Hellman Ephemeral)

1. Both sides agree on an elliptic curve (e.g., X25519, P-256).
2. Client generates random private key c , computes public key $C = c \cdot G$ (G = generator point).
3. Server generates random private key s , computes $S = s \cdot G$.
4. They exchange public keys in the ClientHello and ServerHello.
5. Client computes shared secret: $c \cdot S = c \cdot s \cdot G$.
6. Server computes same: $s \cdot C = s \cdot c \cdot G$. Both arrive at the same secret.
7. Session keys are derived from this shared secret + the random nonces using HKDF.

Ephemeral: New key pair generated for *every* session. If the server's long-term private key is later compromised, past sessions cannot be decrypted (**Perfect Forward Secrecy**).

Why not just RSA?: RSA key exchange sends the premaster secret encrypted with the server's RSA public key. If the server's private key is compromised later, an attacker who recorded the traffic can retroactively decrypt all past sessions. No forward secrecy.

7.2.2 Certificate Chain of Trust

Concept / Pattern

How Certificate Verification Works

1. Server sends its **leaf certificate**: contains domain name(s) (SAN fields), public key, issuer, expiry date, serial number, signature.
2. Leaf certificate is signed by an **Intermediate CA's** private key.
3. Intermediate CA certificate is signed by a **Root CA's** private key.
4. Root CA certificate is **self-signed** and pre-installed in the OS/browser's trust store (Chrome, Firefox have their own).
5. Client verifies the entire chain:
 - Leaf signature valid with Intermediate CA's public key?
 - Intermediate signature valid with Root CA's public key?
 - Root CA in trust store?
 - Domain in cert's SAN (Subject Alternative Name) matches the requested hostname?
 - Certificate not expired?
 - Certificate not revoked (OCSP/CRL check)?

OCSP (Online Certificate Status Protocol): Client queries the CA's OCSP responder to check if a cert is revoked. Adds latency and a privacy leak (CA learns which sites you visit).

OCSP Stapling: Server pre-fetches the OCSP response (valid for 24h), signs it, and includes it in the TLS handshake. Client gets freshness proof without an extra round trip.

Certificate Transparency (CT): All public TLS certs must be logged in public CT logs (run by Google, Cloudflare, DigiCert). Browsers require SCTs (Signed Certificate Timestamps) in the cert or TLS extension. Prevents misissued certs from going undetected.

7.2.3 TLS Cipher Suites

TLS 1.3 reduced cipher suites to 5 (all using AEAD):

- TLS_AES_128_GCM_SHA256
- TLS_AES_256_GCM_SHA384
- TLS_CHACHA20_POLY1305_SHA256 (better for mobile/IoT without hardware AES)
- TLS_AES_128_CCM_SHA256
- TLS_AES_128_CCM_8_SHA256

TLS 1.2 had 37 cipher suites, many now known to be weak (RC4, 3DES, DH non-ephemeral, MD5, SHA-1).

7.3 TLS 1.3 vs TLS 1.2

Feature	TLS 1.2	TLS 1.3
Handshake latency	2-RTT	1-RTT (0-RTT for resumption)
Key exchange	RSA or DH (non-ephemeral allowed)	Only ECDHE (always ephemeral)
Perfect Forward Secrecy	Optional (configurable)	Mandatory
Cipher suites	37 options (many weak)	5 AEAD-only suites
Session resumption	Session ID or session tickets	PSK (pre-shared key)
Downgrade protection	Weak (POODLE, BEAST attacks)	Strong (prevents rollback)
Encrypted handshake	Partial (cert in cleartext in 1.2)	Most of handshake encrypted
Middlebox inspection	Possible (common in enterprises)	Much harder

Interview Corner

Q: What is a man-in-the-middle attack on TLS and how does TLS prevent it?

A MITM attacker intercepts the connection and presents their own certificate (signed by themselves) to the client, while connecting to the real server. The client would see a “Certificate not trusted” error because the attacker’s cert is not signed by a trusted CA.

Unless: (1) the user clicks through the warning, (2) the attacker has compromised a trusted CA and obtained a fraudulent cert for your domain, (3) the user has been convinced to install the attacker’s root CA (common in enterprise proxies for TLS inspection).

Q: Explain Perfect Forward Secrecy.

With RSA key exchange (no PFS): if the server’s private key is compromised, an attacker who recorded past encrypted sessions can now decrypt them all (they can decrypt the premaster secret from the past ClientKeyExchange message).

With ECDHE (PFS): each session uses a freshly generated ephemeral key pair. The ephemeral private key is deleted after the session. Even if the server’s long-term private key is later compromised, past session keys cannot be reconstructed — they depended on ephemeral keys that no longer exist.

Chapter 8

Routing and Routing Protocols

8.1 How Packet Forwarding Works

A router maintains a **Forwarding Information Base (FIB)** derived from the Routing Information Base (RIB). When a packet arrives:

1. Read destination IP from the packet header.
2. Perform **Longest Prefix Match (LPM)** against all entries in the FIB.
3. Decrement TTL. If $TTL = 0$: drop + send ICMP Time Exceeded.
4. Update checksum (TTL changed checksum).
5. Determine outgoing interface and next-hop MAC (via ARP/NDP).
6. Forward the packet. If no match: drop + send ICMP Unreachable.

Concept / Pattern

Longest Prefix Match (LPM) Example

FIB:

10.0.0.0/8	via eth0	(metric 100)
10.10.0.0/16	via eth1	(metric 100)
10.10.5.0/24	via eth2	(metric 100)
0.0.0.0/0	via eth3	(default route)

Destination: 10.10.5.100

Matching prefixes: 10.0.0.0/8 (/8 match), 10.10.0.0/16 (/16 match), 10.10.5.0/24 (/24 match)

LPM chooses: 10.10.5.0/24 (24 bits = most specific). Forward via **eth2**.

For destination 8.8.8.8: Only 0.0.0.0/0 matches. Forward via **eth3** (default route).

Deep Dive / Detail

Hardware LPM: TCAM (Ternary Content Addressable Memory)

Software LPM (trie-based) is $O(\log n)$ per lookup. At 100 Gbps with 1500B packets, that's ~8 million packets/second — software can't keep up.

High-speed routers use **TCAM**: lookup tables where each cell can store 0, 1, or X (don't care). Routing entries are stored as bitmask + value pairs. The lookup is done *in a single hardware clock cycle* (truly $O(1)$) by comparing the destination IP against all entries simultaneously in parallel hardware. Extremely fast but expensive and power-hungry (limits table size to ~1M entries on high-end gear).

8.2 Routing Protocols Classification

Category	Examples	Use
IGP (Interior Gateway)	OSPF, IS-IS, EIGRP, RIP	Within an Autonomous System
EGP (Exterior Gateway)	BGP	Between Autonomous Systems
Distance-Vector	RIP, EIGRP	Share routing table with neighbours
Link-State	OSPF, IS-IS	Flood topology info; each runs Dijkstra
Path-Vector	BGP	Share full AS path; prevents routing loops

8.2.1 RIP: Routing Information Protocol (Historical)

- Distance-vector. Metric = hop count. Max 15 hops (16 = infinity = unreachable).
- Sends full routing table to neighbours every 30s. Slow convergence (minutes).
- **Count-to-infinity problem:** Routing loops can form when a link fails; counters increment slowly to 16 before the route is declared unreachable. Mitigated by split horizon and poison reverse.
- **Split horizon:** Don't advertise a route back to the interface you learned it from.
- **Poison reverse:** Advertise a route with metric 16 (infinity) back toward where you learned it.
- RIPv1: Classful (no subnet info). RIPv2: CIDR, MD5 authentication. RIPng: IPv6.
- Only useful for very small, simple networks. Essentially obsolete.

8.2.2 OSPF: Open Shortest Path First

Concept / Pattern

OSPF Key Concepts (RFC 2328 for OSPFv2, RFC 5340 for OSPFv3)

- **Link-State:** Each router learns the *complete network topology* via LSAs (Link State Advertisements) flooded to all routers in the same area. Each router independently runs Dijkstra's algorithm to compute shortest paths.
- **Cost metric:** Each link has a configurable cost (typically $10^8/\text{bandwidth-bps}$). OSPF finds the minimum-cost path. 100 Mbps = cost 1. 10 Mbps = cost 10. GigE = cost 1 (same as 100M — configure manually!).
- **Area hierarchy:** Areas limit LSA flooding scope. All areas must connect to **Area 0 (backbone)**. ABRs (Area Border Routers) summarise routes between areas.
- **DR/BDR election:** On broadcast networks (Ethernet), elect one **Designated Router** and a **Backup DR**. All others form adjacencies only with DR/BDR (not full mesh), reducing LSA flooding overhead.
- **LSA Types:** Type 1 (Router LSA), Type 2 (Network LSA), Type 3 (Summary LSA), Type 5 (AS-External LSA from ASBRs).
- **OSPF messages:** Hello (neighbour discovery + keepalive), DBD (Database Description, LSA headers), LSR (Link State Request), LSU (Link State Update), LSAck (acknowledgement).
- **Convergence:** Fast (seconds). Link state change triggers immediate LSA flood + Dijkstra re-computation.
- **Authentication:** Supports MD5/SHA-HMAC per-interface to prevent rogue routers injecting LSAs.

8.2.3 BGP: Border Gateway Protocol

BGP (RFC 4271) is the routing protocol of the Internet. It is the only EGP in use, connecting **Autonomous Systems (ASes)**.

Deep Dive / Detail

BGP Deep Dive

- **Autonomous System (AS):** A network under single administrative control with a consistent routing policy (an ISP, a cloud provider, a large enterprise). Each AS has a globally unique ASN (2-byte 1–65535 or 4-byte 1–4294967295, RFC 6793). Google = AS15169. Cloudflare = AS13335. Private ASN range: 64512–65534 (2B), 4200000000–4294967294 (4B).
- **Path-Vector protocol:** BGP advertises IP prefixes with the full AS_PATH (sequence of ASes the route traversed). Loops prevented by rejecting routes containing your own ASN.
- **iBGP vs eBGP:** iBGP = BGP session within the same AS (needs full mesh or Route Reflectors). eBGP = BGP between different ASes. eBGP routes are preferred over iBGP routes. eBGP sets local-pref; iBGP distributes it.
- **BGP attributes:** ORIGIN (i = IGP, e = EGP, ? = incomplete), AS_PATH, NEXT_HOP, LOCAL_PREF (inbound traffic preference; higher = preferred; not sent eBGP), MED/METRIC (metric to next-hop AS; lower = preferred).
- **Best path selection (13 steps):** Local preference → Locally originated → Shortest AS_PATH → Lowest ORIGIN → Lowest MED → eBGP over iBGP → Lowest IGP metric to next-hop → Oldest eBGP route → Lowest Router ID → Lowest neighbour IP.
- **BGP session:** TCP port 179. BGP is reliable (uses TCP) and stateful.
- **Route Reflectors:** In a large AS, full-mesh iBGP is unscalable ($n(n-1)/2$ sessions). Route Reflectors are designated iBGP speakers that reflect routes to clients, without full mesh.
- **BGP Communities:** Well-known or private 32-bit tags attached to routes. Used for routing policies: “do not export”, “low preference”, “black hole”.

Common Mistake / Gotcha

BGP Hijacking — Real Incidents

Attack: An AS announces a more specific prefix for a victim’s IP space (e.g., 8.8.8.0/24 instead of 8.8.0.0/16). LPM selects the more specific route: traffic meant for the victim is now routed to the attacker.

Real incidents: Pakistan Telecom hijacked YouTube in 2008 (18 minutes). Amazon Route 53 hijacked for MyEtherWallet cryptocurrency theft in 2018. Rostelecom injected BGP routes for AWS, Google, Cloudflare (April 2020).

Mitigation: RPKI (Resource Public Key Infrastructure): Cryptographic attestation that an AS is authorised to originate a prefix. ROAs (Route Origin Authorisations) are signed by the address holder and published in a certificate database. BGP routers with RPKI validation drop routes with invalid origin ASNs. Adoption growing but not universal (~50% of prefixes have ROAs).

BGPsec: Extends RPKI to cryptographically sign the full AS_PATH (prevents path manipulation). Computationally expensive; not widely deployed.

8.3 VLANs: Virtual LANs

VLANs (IEEE 802.1Q) allow a single physical switch to behave as multiple independent logical switches, isolating Layer 2 broadcast domains.

Concept / Pattern**802.1Q VLAN Tagging**

A 4-byte tag is inserted into the Ethernet frame between Src MAC and EtherType:

Dst MAC	Src MAC	TPID(2B)	TCI(2B)	EtherType	Payload	FCS
---------	---------	-----------------	----------------	-----------	---------	-----

- **TPID** (Tag Protocol ID, 16b): 0x8100 = 802.1Q tag. QinQ uses 0x88A8.
- **PCP** (Priority Code Point, 3b): 802.1p QoS, 0–7 (7=highest). Used for traffic prioritisation.
- **DEI** (Drop Eligible Indicator, 1b): Mark for preferential dropping under congestion.
- **VID** (VLAN ID, 12b): 0–4095. 0 = priority only. 1 = default. 4095 = reserved.

Access port: Connected to end devices. Switch adds/strips VLAN tag transparently. Device is unaware.

Trunk port: Between switches or switch-router. Carries frames for multiple VLANs with tags intact.

Native VLAN: Untagged traffic on a trunk port belongs to the native VLAN (default: VLAN 1).

Inter-VLAN routing: VLANs are separate L2 domains; need a L3 device to route between them:

- **Router-on-a-stick:** Single router interface with sub-interfaces (one per VLAN), each with an IP. Trunk port to switch. Bottleneck.
- **Layer 3 switch:** Routes between VLANs at wire speed using hardware. SVI (Switched Virtual Interface) per VLAN acts as the gateway.

8.4 MPLS: Multiprotocol Label Switching

MPLS (RFC 3031) is a packet-forwarding mechanism used in ISP backbones. Instead of IP-level routing lookups at each hop, packets are assigned a 32-bit **label** at the network edge. Core routers forward based on labels alone (extremely fast table lookup).

- **Label stack:** Multiple labels can be stacked (MPLS-over-MPLS for VPNs).
- **LER (Label Edge Router):** Ingress: assigns label based on IP destination + VPN/QoS policy. Egress: removes label, delivers IP packet.
- **LSR (Label Switching Router):** Core router. Swaps incoming label with outgoing label. No IP lookup.
- **MPLS VPN (L3VPN):** Enterprises use MPLS VPNs from ISPs. Each customer VPN gets its own label space. PE (Provider Edge) routers run BGP to exchange customer routes.
- **Traffic Engineering (MPLS-TE):** Force traffic onto specific pre-computed paths (LSPs), bypassing normal IP routing. Useful for bandwidth guarantees and avoiding congested links.
- **Decline:** MPLS is being displaced by SD-WAN and SASE (cloud-based networking). Many enterprises are moving off MPLS to internet-based connectivity.

Chapter 9

Network Security

9.1 Common Network Attacks

9.1.1 TCP SYN Flood

Deep Dive / Detail

Attack: Attacker sends many SYN packets with spoofed source IPs. Server allocates a TCB (Transmission Control Block) for each, filling the SYN backlog queue. Legitimate connections cannot complete the 3-way handshake.

SYN Cookies (RFC 4987): Instead of allocating state on SYN, encode connection info (src IP, dst IP, src port, dst port, timestamp, ISN) as a cryptographic hash in the SYN-ACK's ISN. State is only allocated when a valid ACK arrives (proving the client received the SYN-ACK — impossible with a spoofed IP).

Enable on Linux: `sysctl net.ipv4.tcp_syncookies=1`.

Trade-off: SYN cookies can't support all TCP options (SACK, window scale, timestamps) since those options need to be remembered from the SYN, which is not stored.

9.1.2 DDoS: Distributed Denial of Service

Deep Dive / Detail

Attack Categories:

Volumetric attacks: Flood the target with more bandwidth than it can absorb. DNS amplification (70x amplification), NTP amplification (556x), SSDP amplification. Source IP spoofing makes the amplifier send replies to the victim.

Protocol attacks: Exhaust stateful resources. SYN flood (fills connection table), ICMP flood, BGP session exhaustion.

Application layer attacks: HTTP floods (appear legitimate), Slowloris (hold connections open by sending headers very slowly, never completing request), ReDoS (regex denial of service).

Mitigations:

- **Rate limiting:** Per-IP or per-subnet request rate limits (iptables, Nginx, WAF).
- **IP reputation:** Block known bad IPs using threat intelligence feeds.
- **BGP Blackholing (RTBH):** Announce the victim's IP with a "null route" community. Traffic is discarded at the upstream ISP's routers before reaching your network. Effective but the victim is still unreachable (it's DoS with a different address).
- **Anycast absorption:** Route traffic through DDoS scrubbing centres (Cloudflare absorbs 100 Tbps+).
- **BCP38:** ISPs filter packets with spoofed source IPs at the network edge. If universally deployed, amplification attacks would be impossible. Adoption is partial.

9.1.3 MITM Attacks

ARP Poisoning MITM: Attacker poisons ARP caches on the LAN, redirecting traffic through themselves.

DNS Spoofing MITM: Attacker injects false DNS responses, redirecting victims to a malicious server.

BGP Hijacking MITM: Attacker announces victim's prefix, receiving traffic meant for the victim.

SSL Stripping: Attacker intercepts HTTPS redirect and serves HTTP, preventing TLS.

Defences:

- **TLS:** Encrypts and authenticates. MITM requires forging a certificate (needs to compromise a CA).
- **HSTS (HTTP Strict Transport Security):** Once a browser sees the HSTS header, it refuses HTTP and rejects invalid TLS certs for that domain. **Strict-Transport-Security:** `max-age=31536000; includeSubDomains; preload`
- **HSTS Preload:** Browsers ship with a hardcoded list of HSTS domains. Even the very first connection is HTTPS. Submit at hstspreload.org.
- **Certificate Pinning:** Application hardcodes the expected cert or public key. Rejects valid-CA-signed certs that don't match. Used in mobile apps but risky (breaks on cert renewal).
- **DAI (Dynamic ARP Inspection):** Switch validates ARP packets against DHCP snooping binding table.

9.2 Firewalls

Concept / Pattern

Firewall Types and Capabilities

- **Stateless (Packet Filter)**: Inspects each packet independently. Rules: src/dst IP, port, protocol. Fast but limited. Cannot track TCP state (can't distinguish a new connection from an existing one without looking at flags manually). Examples: BSD pf, early iptables.
- **Stateful**: Maintains a connection tracking table. Understands TCP state machine (SYN, ESTABLISHED, FIN_WAIT...). Only allows return traffic for established outgoing connections. Default for all modern firewalls.
- **Application (L7) / WAF (Web Application Firewall)**: Understands HTTP, SQL, etc. Detects: SQL injection in query params, XSS payloads, malformed HTTP, bot fingerprints, OWASP Top 10 attacks. Examples: ModSecurity, AWS WAF, Cloudflare WAF.
- **NGFW (Next-Gen Firewall)**: Combines stateful + application inspection + IDS/IPS + deep packet inspection + TLS inspection (decrypt-inspect-re-encrypt) + user identity awareness. Examples: Palo Alto, Fortinet, Cisco Firepower.

Example / Code

Linux iptables: Building a Stateful Firewall

```
# Default policies: drop everything not explicitly allowed
iptables -P INPUT DROP
iptables -P FORWARD DROP
iptables -P OUTPUT ACCEPT # Allow all outgoing (tighten in production)

# Allow loopback
iptables -A INPUT -i lo -j ACCEPT

# Allow established/related traffic (stateful; handles TCP replies, ICMP errors)
iptables -A INPUT -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT

# Allow SSH (brute-force protection: max 5 new connections per minute per IP)
iptables -A INPUT -p tcp --dport 22 -m conntrack --ctstate NEW \
  -m recent --set --name SSH --rsource
iptables -A INPUT -p tcp --dport 22 -m conntrack --ctstate NEW \
  -m recent --update --seconds 60 --hitcount 5 --name SSH --rsource -j DROP
iptables -A INPUT -p tcp --dport 22 -j ACCEPT

# Allow HTTP and HTTPS
iptables -A INPUT -p tcp -m multiport --dports 80,443 -j ACCEPT

# Allow ICMP ping (with rate limit)
iptables -A INPUT -p icmp --icmp-type echo-request -m limit \
  --limit 5/s --limit-burst 10 -j ACCEPT

# Log and drop everything else
iptables -A INPUT -j LOG --log-prefix "iptables-drop: " --log-level 4
iptables -A INPUT -j DROP

# Save rules (Debian/Ubuntu)
iptables-save > /etc/iptables/rules.v4
```

9.3 VPN: Virtual Private Networks

Deep Dive / Detail

VPN Technologies Comparison

IPsec: Network layer encryption. Two modes:

- **Transport mode:** Encrypts/authenticates only the payload. IP header unchanged.
- **Tunnel mode:** Wraps the entire IP packet in a new IP packet with new header. Used for site-to-site VPNs.

IPsec protocols: **AH** (Authentication Header, integrity only), **ESP** (Encapsulating Security Payload, encryption + integrity). Uses IKE (Internet Key Exchange) for key negotiation on UDP port 500/4500.

OpenVPN: TLS-based VPN over UDP or TCP. Highly flexible, cross-platform. Slow due to userspace implementation.

WireGuard: Modern VPN using state-of-the-art cryptography (Curve25519 ECDH, ChaCha20, BLAKE2s). Extremely fast (1/5 of OpenVPN CPU usage), small codebase (4000 lines vs OpenVPN's 100K+), kernel module (Linux 5.6+). Uses UDP. No negotiation: peers are pre-configured. Stateless handshake.

SSL/TLS VPN (e.g., GlobalProtect, AnyConnect): User connects via HTTPS (port 443). Very firewall-friendly (port 443 is rarely blocked). Works through restrictive networks. Used for remote access.

9.4 Network Scanning and Intrusion Detection

Example / Code

nmap: Network Discovery and Security Auditing

```
# Host discovery (ping sweep without port scan)
nmap -sn 192.168.1.0/24

# TCP SYN scan (stealthy, doesn't complete 3-way handshake)
nmap -sS 192.168.1.1

# Full port scan (all 65535 ports)
nmap -p- 192.168.1.1

# Service and version detection
nmap -sV -sC 192.168.1.1      # -sC: run default scripts

# OS fingerprinting
nmap -O 192.168.1.1

# Aggressive scan (OS, version, scripts, traceroute)
nmap -A 192.168.1.1

# Scan multiple targets
nmap -sS 10.0.0.0/8 -p 22,80,443 --open # Show only open ports

# Firewall evasion: fragment packets
nmap -f 192.168.1.1      # Split into 8-byte IP fragments

# UDP scan (slow, requires root)
nmap -sU -p 53,67,68,123,161 192.168.1.1
```

Chapter 10

Socket Programming and Network I/O

10.1 The Socket API: Fundamental Concepts

A **socket** is a kernel abstraction representing a network communication endpoint, identified by:

- **Socket address:** (IP address, port number, protocol).
- **TCP connection:** Identified by a *4-tuple*: (src_ip, src_port, dst_ip, dst_port). These 4 values together uniquely identify a connection on any host.
- **Socket descriptor:** A file descriptor (integer) used to interact with the socket from application code.

Deep Dive / Detail

Port Number Ranges

- **Well-known ports (0–1023):** Reserved for standard protocols. Require root/admin. HTTP=80, HTTPS=443, SSH=22, SMTP=25, DNS=53, FTP=20/21, NTP=123.
- **Registered ports (1024–49151):** IANA-registered for specific applications. MySQL=3306, PostgreSQL=5432, Redis=6379, MongoDB=27017.
- **Ephemeral ports (49152–65535):** Assigned by the OS to clients for outgoing connections. Range configurable: `/proc/sys/net/ipv4/ip_local_port_range`.

Example / Code

Complete TCP Server and Client (C)

```

1  /* === TCP SERVER === */
2  #include <stdio.h>
3  #include <stdlib.h>
4  #include <string.h>
5  #include <unistd.h>
6  #include <arpa/inet.h>
7  #include <sys/socket.h>
8  #include <netinet/in.h>
9  #include <netinet/tcp.h>
10
11 int create_server(uint16_t port) {
12     // AF_INET = IPv4, SOCK_STREAM = TCP, 0 = auto-select protocol
13     int fd = socket(AF_INET, SOCK_STREAM, 0);
14     if (fd < 0) { perror("socket"); exit(1); }
15
16     // SO_REUSEADDR: Bind to port even if in TIME_WAIT (essential for server restart)
17     int opt = 1;
18     setsockopt(fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));
19
20     // SO_REUSEPORT: Multiple sockets on same port (used by nginx for multi-worker)
21     setsockopt(fd, SOL_SOCKET, SO_REUSEPORT, &opt, sizeof(opt));
22
23     struct sockaddr_in addr = {
24         .sin_family      = AF_INET,
25         .sin_port        = htons(port),          // host-to-network short (byte order)
26         .sin_addr.s_addr = INADDR_ANY,          // bind to all interfaces (0.0.0.0)
27     };
28     if (bind(fd, (struct sockaddr *)&addr, sizeof(addr)) < 0) {
29         perror("bind"); exit(1);
30     }
31
32     // SOMAXCONN: max backlog (SYN_RCVD + ESTABLISHED not yet accept()ed)
33     // Increase /proc/sys/net/core/somaxconn for high-load servers
34     if (listen(fd, SOMAXCONN) < 0) { perror("listen"); exit(1); }
35
36     printf("Listening on port %d\n", port);
37     return fd;
38 }
39
40 void handle_client(int client_fd, struct sockaddr_in *addr) {
41     char ip[INET_ADDRSTRLEN];
42     inet_ntop(AF_INET, &addr->sin_addr, ip, sizeof(ip));
43     printf("Client: %s:%d\n", ip, ntohs(addr->sin_port));
44
45     // Disable Nagle for low-latency response
46     int flag = 1;
47     setsockopt(client_fd, IPPROTO_TCP, TCP_NODELAY, &flag, sizeof(flag));
48
49     char buf[65536];
50     ssize_t n;
51     // Read returns 0 on EOF (client closed), -1 on error
52     while ((n = read(client_fd, buf, sizeof(buf))) > 0) {
53         if (write(client_fd, buf, n) != n) break; // echo back
54     }
55     close(client_fd);
56     printf("Client disconnected\n");
57 }
58
59 int main() {
60     int server_fd = create_server(8080);
61     while (1) {
62         struct sockaddr_in client_addr;
63         socklen_t addr_len = sizeof(client_addr);
64         int client_fd = accept(server_fd, (struct sockaddr *)&client_addr, &addr_len);
65         if (client_fd < 0) { perror("accept"); continue; }
66         handle_client(client_fd, &client_addr); // single-threaded; blocks here
67     }
68 }

```

Example / Code

TCP Client (C)

```
1 #include <stdio.h>
2 #include <string.h>
3 #include <unistd.h>
4 #include <arpa/inet.h>
5 #include <sys/socket.h>
6
7 int main() {
8     int fd = socket(AF_INET, SOCK_STREAM, 0);
9
10    struct sockaddr_in server = {
11        .sin_family = AF_INET,
12        .sin_port   = htons(8080),
13    };
14    inet_pton(AF_INET, "127.0.0.1", &server.sin_addr); // text to binary IP
15
16    if (connect(fd, (struct sockaddr *)&server, sizeof(server)) < 0) {
17        perror("connect"); return 1;
18    }
19
20    const char *msg = "Hello, server!\n";
21    write(fd, msg, strlen(msg));
22
23    char buf[1024] = {};
24    ssize_t n = read(fd, buf, sizeof(buf));
25    if (n > 0) printf("Reply: %.*s", (int)n, buf);
26
27    close(fd);
28    return 0;
29 }
```

10.2 Key Socket Options Reference

Option	Effect and Notes
SO_REUSEADDR	Bind to port in TIME_WAIT. Essential for server restart without waiting.
SO_REUSEPORT	Multiple sockets bind to same port. OS load-balances incoming connections. Used by nginx multi-worker model.
SO_KEEPALIVE	Send keepalive probes after idle period. Detects dead connections. Configure interval with TCP_KEEPIDLE, TCP_KEEPINTVL, TCP_KEEPCNT.
SO_RCVBUF	Set receive buffer size. Affects rwnd. Kernel doubles the value set.
SO_SNDBUF	Set send buffer size. Limits how much unsent data can be buffered.
SO_LINGER	If linger=0: <code>close()</code> sends RST (immediate abort). If linger=N: <code>close()</code> blocks up to N seconds waiting for data to be sent.
SO_RCVTIMEO	Receive timeout (<code>struct timeval</code>). <code>recv()</code> returns EAGAIN after timeout.
SO_SNDTIMEO	Send timeout.
TCP_NODELAY	Disable Nagle's algorithm. Send immediately. Reduces latency. Essential for Redis, SSH, games.
TCP_CORK	Buffer all data until MSS is full or uncorked. Batches headers with data. Combine with <code>sendfile()</code> .
TCP_FASTOPEN	Enable TCP Fast Open. Server: <code>setsockopt</code> enables TFO. Client: <code>sendto()</code> in SYN.
TCP_KEEPIDLE	Seconds idle before first keepalive probe (default: 7200s = 2h).
TCP_KEEPINTVL	Interval between keepalive probes (default: 75s).
TCP_KEEPCNT	Number of probes before declaring connection dead (default: 9).
IP_TOS	Set the DSCP/ToS byte in the IP header for QoS marking.
IP_TTL	Override default TTL (64) for this socket.
SO_BINDTODEVICE	Bind socket to a specific network interface.

10.3 I/O Multiplexing: Handling Thousands of Connections

A blocking server handles one connection at a time. Real servers handle thousands to millions of concurrent connections. This requires **I/O multiplexing**.

10.3.1 The C10K and C10M Problem

- **C10K (circa 2000)**: Achieve 10,000 concurrent connections on a single server. The solution was event-driven I/O (`select`, `poll`, `epoll`). Apache's thread-per-connection model failed; nginx's event-driven model succeeded.
- **C10M (2012)**: 10 million concurrent connections. Requires careful OS tuning, zero-copy I/O, userspace networking stacks (DPDK).

10.3.2 `select()` and `poll()`

`select()`: Pass sets of file descriptors; it returns when any is readable/writable/error.

Limitations: fd limit = 1024 (FD_SETSIZE). $O(n)$ scan — kernel scans all fds even if only 1 is ready. Fds copied to kernel on every call.

`poll()`: Like `select()` but uses an array, no 1024 limit. Still $O(n)$ and copies fd array every call.

10.3.3 epoll: The Linux Standard for High-Performance I/O

Example / Code

epoll Event Loop (Edge-Triggered, High-Performance Pattern)

```

1 #include <sys/epoll.h>
2 #include <fcntl.h>
3
4 #define MAX_EVENTS 512
5
6 static void set_nonblocking(int fd) {
7     int flags = fcntl(fd, F_GETFL, 0);
8     fcntl(fd, F_SETFL, flags | O_NONBLOCK);
9 }
10
11 int main() {
12     int server_fd = create_server(8080); // from earlier
13     set_nonblocking(server_fd);
14
15     int epfd = epoll_create1(EPOOL_CLOEXEC);
16
17     struct epoll_event ev = {
18         .events = EPOLLIN,           // Level-triggered for accept()
19         .data.fd = server_fd,
20     };
21     epoll_ctl(epfd, EPOLL_CTL_ADD, server_fd, &ev);
22
23     struct epoll_event events[MAX_EVENTS];
24
25     while (1) {
26         // Block until events are ready. -1 = wait forever.
27         int n = epoll_wait(epfd, events, MAX_EVENTS, -1);
28
29         for (int i = 0; i < n; i++) {
30             int fd = events[i].data.fd;
31
32             if (fd == server_fd) {
33                 // New connection (level-triggered, so just one accept per event)
34                 // Use accept4 for SOCK_NONBLOCK flag (atomic)
35                 int client_fd = accept4(server_fd, NULL, NULL, SOCK_NONBLOCK);
36                 if (client_fd < 0) continue;
37
38                 struct epoll_event cev = {
39                     // EPOLLET = edge-triggered: notify ONCE on state change
40                     // EPOLLIN | EPOLLRDHUP: readable + peer shutdown
41                     .events = EPOLLIN | EPOLLET | EPOLLRDHUP,
42                     .data.fd = client_fd,
43                 };
44                 epoll_ctl(epfd, EPOLL_CTL_ADD, client_fd, &cev);
45
46             } else if (events[i].events & (EPOLLRDHUP | EPOLLERR | EPOLLHUP)) {
47                 // Client disconnected or error
48                 epoll_ctl(epfd, EPOLL_CTL_DEL, fd, NULL);
49                 close(fd);
50
51             } else if (events[i].events & EPOLLIN) {
52                 // Edge-triggered: MUST read ALL available data in a loop
53                 char buf[4096];
54                 ssize_t n;
55                 while ((n = read(fd, buf, sizeof(buf))) > 0)
56                     write(fd, buf, n); // echo (real server: handle EAGAIN)
57
58                 if (n == 0) { // EOF
59                     epoll_ctl(epfd, EPOLL_CTL_DEL, fd, NULL);
60                     close(fd);
61                 }
62                 // n == -1 && errno == EAGAIN: all data read, stop loop
63             }
64         }
65     }
66 }

```


epoll advantages:

- $O(1)$ per event regardless of total number of monitored fds.
- Register fds once; receive events indefinitely.
- **Edge-triggered (EPOLLET)**: Notified only when state changes (new data arrives). Must drain all data. More efficient but requires careful coding.
- **Level-triggered (default)**: Notified as long as data is available. Simpler but slightly less efficient.

10.3.4 io_uring (Linux 5.1+, 2019)

io_uring (designed by Jens Axboe) is a revolutionary async I/O interface using two shared ring buffers between kernel and userspace:

- **Submission Queue (SQ)**: Userspace writes I/O operation descriptors.
- **Completion Queue (CQ)**: Kernel writes completion events.
- **No syscall per I/O**: With `IORING_SETUP_SQPOLL`, kernel polls the SQ — zero syscalls for I/O submission.
- Supports all I/O: `read`, `write`, `open`, `close`, `send`, `recv`, `accept`, `connect`, `fsync`, `splice`, `statx`, linked I/O chains.
- **Fixed buffers**: Pre-register buffers to avoid per-operation copy overhead.
- Used by: RocksDB, NGINX 1.25+, systemd, PostgreSQL 13+.

10.3.5 The Reactor and Proactor Patterns

Reactor pattern (epoll/kqueue): The application registers interest in events. When an event fires, the application is called and must perform the I/O synchronously. Used by: Node.js, nginx, Redis.

Proactor pattern (io_uring, Windows IOCP): The application submits an I/O operation. The kernel performs the I/O. The application is notified when it's *done*. More scalable: kernel can use the most efficient method (DMA, zero-copy).

10.4 Non-blocking Sockets and Common Pitfalls

Example / Code

Non-blocking connect() Pattern

```

1 #include <errno.h>
2 #include <string.h>
3 #include <sys/socket.h>
4 #include <fcntl.h>
5
6 // Non-blocking connect: returns immediately; use epoll to wait for EPOLLOUT
7 int nb_connect(const char *ip, uint16_t port) {
8     int fd = socket(AF_INET, SOCK_STREAM | SOCK_NONBLOCK, 0);
9
10    struct sockaddr_in addr = { .sin_family = AF_INET, .sin_port = htons(port) };
11    inet_pton(AF_INET, ip, &addr.sin_addr);
12
13    int rc = connect(fd, (struct sockaddr *)&addr, sizeof(addr));
14    if (rc < 0 && errno == EINPROGRESS) {
15        // Normal for non-blocking connect: wait for EPOLLOUT
16        return fd;
17    }
18    if (rc == 0) return fd; // Connected immediately (loopback)
19    close(fd); return -1; // Error
20 }
21
22 // After EPOLLOUT fires on the fd:
23 int check_connect_result(int fd) {
24     int err = 0;
25     socklen_t len = sizeof(err);
26     if (getsockopt(fd, SOL_SOCKET, SO_ERROR, &err, &len) < 0 || err != 0) {
27         // Connection failed (ECONNREFUSED, ETIMEDOUT, etc.)
28         fprintf(stderr, "connect failed: %s\n", strerror(err));
29         return -1;
30     }
31     return 0; // Connected successfully
32 }

```

Common Mistake / Gotcha

Common Non-blocking I/O Mistakes

- **Not handling EINTR:** `read()`/`write()` can be interrupted by signals. Always retry on `EINTR`.
- **Not handling partial writes:** `write()` on a socket may write fewer bytes than requested. Always check the return value and loop.
- **Confusing EAGAIN with errors:** In non-blocking mode, `EAGAIN` (= `EWOULDBLOCK`) means “no data available right now, try again.” It’s not an error.
- **EPOLLOUT starvation:** If you always have data to write, the fd is always writable. Don’t add `EPOLLOUT` to `epoll` unless you have buffered data to send; remove it once the buffer is drained.
- **Edge-triggered missing events:** If you don’t drain all available data in edge-triggered mode, you won’t get another `EPOLLIN` until new data arrives. Buffer the unread data and resume on the next event.

10.5 WebSocket Protocol

WebSockets (RFC 6455) upgrade an HTTP/1.1 connection to a persistent, full-duplex, message-oriented channel.

Concept / Pattern**WebSocket Handshake and Frame Format**

Client sends HTTP Upgrade request:

```
GET /chat HTTP/1.1
Host: ws.example.com
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Key: dGhlIHNhbXBsZSBub25jZQ==    <- random 16 bytes, base64
Sec-WebSocket-Version: 13
Sec-WebSocket-Extensions: permessage-deflate    <- optional compression
```

Server responds:

```
HTTP/1.1 101 Switching Protocols
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Accept: s3pPLMBiTxaQ9kYGzzhZRbK+x0o=    <- SHA1(key + GUID), base64
```

From this point, the TCP connection carries **WebSocket frames** (binary or text), not HTTP.

Frame format:

- **FIN bit:** Last fragment of a message.
- **Opcode:** 0x0=continuation, 0x1=text, 0x2=binary, 0x8=close, 0x9=ping, 0xA=pong.
- **MASK bit:** Client frames must be masked (prevents cache poisoning attacks on transparent proxies). 4-byte masking key XOR'd with payload.
- **Payload length:** 7-bit (0–125), or 16-bit (126), or 64-bit (127).

Heartbeats: Application-level ping/pong (WebSocket opcodes 0x9/0xA) to detect dead connections, since TCP keepalives may not be set.

Chapter 11

Application Layer Protocols and Architectures

11.1 Email Protocols: SMTP, POP3, IMAP

Deep Dive / Detail

Email Protocol Stack

SMTP (Simple Mail Transfer Protocol, RFC 5321): Used for *sending* email. Server-to-server (MTA to MTA) and client-to-server (MUA to MTA).

- Port 25: MTA-to-MTA (often blocked by ISPs to prevent spam).
- Port 587 (STARTTLS): Client-to-server submission. Authenticated.
- Port 465 (SMTPS): Client-to-server, TLS from the start (implicit TLS).

IMAP (Internet Message Access Protocol, RFC 3501): Client accesses mailbox on server. Messages stay on server; synced across devices. Port 143 (STARTTLS), Port 993 (IMAPS). The modern standard.

POP3 (Post Office Protocol v3): Downloads and deletes messages from server. No sync across devices. Port 110, Port 995 (POP3S). Legacy; still used for simple setups.

Email authentication:

- **SPF (Sender Policy Framework):** TXT record listing IPs authorised to send email for a domain. `v=spf1 include:_spf.google.com all`
- **DKIM (DomainKeys Identified Mail):** Email signed with private key. Public key in DNS TXT record. Verifies email hasn't been modified in transit.
- **DMARC (Domain-based Message Authentication, Reporting & Conformance):** Combines SPF + DKIM. Tells receivers what to do with failed authentication (quarantine/reject). Provides aggregate reports.

11.2 SSH: Secure Shell

SSH (Secure Shell, RFC 4251) is a protocol for secure remote login, file transfer (SFTP/SCP), and port forwarding, over an encrypted tunnel.

Example / Code

SSH Essential Usage and Configuration

```
# Basic remote login
ssh user@hostname
ssh -p 2222 user@hostname          # Custom port
ssh -i ~/.ssh/id_ed25519 user@host # Specify private key

# Key generation (Ed25519 is modern and preferred)
ssh-keygen -t ed25519 -C "your@email.com"
# ~/.ssh/id_ed25519 (private), ~/.ssh/id_ed25519.pub (public)

# Install public key on remote (adds to ~/.ssh/authorized_keys)
ssh-copy-id user@hostname

# Port forwarding
# Local forward: access remote service locally
ssh -L 8080:localhost:80 user@remote # localhost:8080 -> remote:80
# Remote forward: expose local service to remote
ssh -R 9090:localhost:3000 user@remote # remote:9090 -> localhost:3000
# Dynamic SOCKS proxy
ssh -D 1080 user@remote              # Use as SOCKS5 proxy

# SSH config file (~/.ssh/config)
# Host myserver
#   HostName 192.168.1.100
#   User admin
#   Port 2222
#   IdentityFile ~/.ssh/id_ed25519
#   ServerAliveInterval 60

# SCP (secure copy)
scp file.txt user@host:/remote/path/
scp -r dir/ user@host:/remote/      # Recursive
```

Deep Dive / Detail

SSH Security Best Practices

- **Disable password authentication:** Only allow key-based auth (`PasswordAuthentication no` in `/etc/ssh/sshd_config`).
- **Disable root login:** `PermitRootLogin no`. Use a non-root user + `sudo`.
- **Change default port:** Port 2222 reduces automated scanning noise (security through obscurity — not a real defence, but reduces log noise).
- **AllowUsers:** Restrict which users can SSH: `AllowUsers alice bob`.
- **Fail2ban:** Automatically ban IPs after N failed login attempts.
- **Use Ed25519 keys:** Smaller, faster, and more secure than RSA-2048.
- **SSH certificates:** Instead of distributing individual public keys, use a CA to sign user/host keys. Scales much better in large organisations.

11.3 FTP and Its Quirks

FTP (RFC 959) is notable for its unusual design:

- Uses **two TCP connections**: Control (port 21, commands) and Data (separate connection for each file transfer).
- **Active mode**: Server initiates data connection back to client (breaks NAT/firewalls).
- **Passive mode (PASV)**: Client initiates both connections. Server opens a random high port and tells

client to connect to it. NAT-friendly.

- No encryption. Credentials and data sent in cleartext.
- **FTPS**: FTP + TLS. Explicit FTPS (STARTTLS on port 21) or Implicit FTPS (port 990).
- **SFTP (SSH File Transfer Protocol)**: Completely different protocol from FTP. Runs over SSH (port 22). Secure, widely used.
- **Recommendation**: Use SFTP or HTTPS for file transfer. FTP is obsolete.

11.4 REST and gRPC

Concept / Pattern

RESTful API Design Principles (Roy Fielding, 2000)

1. **Stateless**: Each request contains all necessary information. No server-side session between requests.
2. **Client-Server**: Separation of UI from data storage/logic.
3. **Cacheable**: Responses must label themselves as cacheable or not.
4. **Uniform Interface**: Consistent resource identification (URLs), manipulation via representations (JSON/XML), self-descriptive messages, HATEOAS (links to related actions).
5. **Layered System**: Client can't tell if it's talking to origin or a proxy/CDN.
6. **Code on Demand (optional)**: Server can send executable code (JavaScript).

Best REST URL design:

- Resources are nouns: `/users/123/orders` not `/getOrdersByUser?id=123`
- Use HTTP methods for actions: `POST /users` (create), `GET /users/123` (read), `PUT /users/123` (replace), `PATCH /users/123` (partial update), `DELETE /users/123`.
- Return appropriate status codes (201 Created, 404 Not Found, 422 Validation Error).
- Version your API: `/api/v2/users` or `Accept: application/vnd.api+json;version=2`.

	REST + JSON	gRPC + Protobuf
Protocol	HTTP/1.1 or HTTP/2	HTTP/2 required
Payload format	JSON (text, human-readable)	Protocol Buffers (binary)
Payload size	Larger	3 – 10× smaller
Speed	Slower (text parsing)	~7x faster serialisation
Streaming	SSE (server), WebSocket (bidirectional)	Native bidirectional streaming
Browser support	Universal	Limited (needs grpc-web proxy)
Debugging	Easy (curl, browser)	Harder (binary format)
Schema	OpenAPI/Swagger (optional)	Proto file (required, enforced)
Code generation	Optional	Built-in for 10+ languages
Use case	Public APIs, web apps	Internal microservices

11.5 CDNs: Content Delivery Networks

Concept / Pattern**How CDNs Work**

1. Domain owner points `cdn.example.com` to a CDN via DNS CNAME or NS delegation.
2. CDN has **PoPs (Points of Presence)** in hundreds of cities worldwide.
3. DNS resolver returns the IP of the nearest PoP (via **anycast** routing or GeoDNS).
4. The PoP checks its cache: **HIT** → respond instantly (edge latency <5ms). **MISS** → fetch from origin (“cache fill”), cache it, respond.
5. Subsequent requests to this PoP are served from cache until TTL expires.

Cache invalidation strategies:

- **URL-based:** Hash of URL determines cache key. Use **Cache-Control** TTL.
- **Purge API:** Instantly invalidate specific URLs or patterns via CDN API.
- **Versioned URLs:** `/app.a1b2c3.js`. Change the URL to bust the cache.
- **Stale-while-revalidate:** Serve stale content while fetching fresh in background.

Edge computing: Modern CDNs (Cloudflare Workers, AWS Lambda@Edge, Fastly Compute) run code at each PoP. Use cases: auth at the edge, A/B testing, personalisation, geo-blocking, request routing.

11.6 Load Balancing

Concept / Pattern**Layer 4 vs Layer 7 Load Balancing**

L4 LB (TCP/UDP level): Routes based on IP addresses and TCP/UDP ports. No understanding of HTTP. Very fast (kernel-level processing, no TLS termination required). Cannot inspect URL, headers, or cookies. **ECMP (Equal-Cost Multipath):** Multiple routes to the same destination; each packet hashed to a different path.

L7 LB (Application level): Terminates TLS. Reads HTTP headers, URL, cookies. Can route based on URL path (`/api` to one backend, `/static` to another), Host header, user identity, content type. Supports sticky sessions, A/B testing, health checks, request buffering, compression. Examples: nginx, HAProxy, AWS ALB, Envoy.

Deep Dive / Detail**Load Balancing Algorithms**

- **Round Robin:** Distribute in order. Simple. Ignores server capacity/load. Good when requests have uniform cost.
- **Weighted Round Robin:** Servers with higher weight receive proportionally more requests. Use when servers have different capacities.
- **Least Connections:** New request goes to server with fewest active connections. Best when requests have variable duration (some are fast, some slow).
- **Least Response Time:** Factor both connection count and measured latency/response time.
- **IP Hash:** Client IP determines the server (sticky, no cookie needed). Same client always hits same server. Breaks down when servers added/removed (use consistent hashing instead).
- **Random with Power of Two Choices:** Pick 2 random servers, send to the one with fewer connections. Near-optimal performance with minimal coordination overhead ($O(1)$). Used by nginx and Google.
- **Consistent Hashing:** Servers and requests mapped to a ring. Each request goes to the nearest server clockwise. Adding/removing a server remaps only $1/n$ of requests (vs. $n/(n+1)$ for simple modulo hashing). Used in: Redis Cluster, Cassandra, Memcached, CDN edge caches.

Chapter 12

Wireless Networks

12.1 IEEE 802.11 Wi-Fi Standards

Standard	Name	Freq	Max Speed	Range	Year
802.11b	Wi-Fi 1	2.4 GHz	11 Mbps	35m	1999
802.11g	Wi-Fi 2	2.4 GHz	54 Mbps	38m	2003
802.11n	Wi-Fi 4	2.4/5 GHz	600 Mbps	70m	2009
802.11ac	Wi-Fi 5	5 GHz	3.5 Gbps	35m	2013
802.11ax	Wi-Fi 6/6E	2.4/5/6 GHz	9.6 Gbps	30m	2019
802.11be	Wi-Fi 7	2.4/5/6 GHz	46 Gbps	30m	2024

12.1.1 CSMA/CA: Collision Avoidance

Wi-Fi uses **CSMA/CA (Carrier Sense Multiple Access with Collision Avoidance)**, not CSMA/CD (Collision Detection like Ethernet). Collision detection is not feasible in wireless (can't transmit and listen simultaneously due to signal strength difference).

Concept / Pattern

CSMA/CA Process

- Sense the channel:** If idle, wait for a DIFS (Distributed Interframe Space, 34 μ s) period.
- Random backoff:** Choose a random backoff counter from 0 to CW (contention window). Count down while channel is idle. When counter reaches 0, transmit.
- Wait for ACK:** After transmitting, wait for an ACK from the AP. If no ACK (collision or loss), double the CW (binary exponential backoff) and retry.
- ACK received:** Transmission successful. Reset CW to minimum.

RTS/CTS (Request-to-Send / Clear-to-Send): Optional mechanism to reserve the channel before transmitting large frames. Reduces collisions (hidden node problem). Overhead for small frames.

Hidden node problem: Station A and Station C can both communicate with AP B but can't hear each other. Both sense the channel as idle and transmit simultaneously $\rightarrow$ collision at AP. RTS/CTS solves this.

12.1.2 Wi-Fi 6 (802.11ax) Key Improvements

- OFDMA:** Orthogonal Frequency Division Multiple Access. Divides a channel into smaller resource units (RUs) allocated to different clients simultaneously (uplink and downlink). Reduces latency, improves efficiency in dense environments.
- MU-MIMO:** Multi-User Multiple Input Multiple Output. Up to 8 simultaneous spatial streams (Wi-Fi 5 had 4 downlink MU-MIMO, no uplink MU-MIMO).

- **BSS Colouring**: Spatial reuse. Transmissions from a different BSS (access point) with a different colour can proceed simultaneously if signal is below a threshold.
- **TWT (Target Wake Time)**: Devices negotiate when to wake up to receive data. IoT devices sleep longer, dramatically reducing power consumption.
- **1024-QAM**: Higher modulation for better throughput when SNR is high.

12.2 Wi-Fi Security

Deep Dive / Detail

Wi-Fi Security Standards Evolution

WEP (Wired Equivalent Privacy): First Wi-Fi security. RC4 encryption with 40-bit or 104-bit keys. Completely broken by 2001. FMS attack, chopchop attack. Never use.

WPA (Wi-Fi Protected Access): TKIP (Temporal Key Integrity Protocol) encryption. Transitional standard. Also broken. Never use.

WPA2-Personal (PSK): AES-CCMP encryption (AES-128 + CBC-MAC). PBKDF2 derives keys from passphrase + SSID. Vulnerable to: offline dictionary attacks on the 4-way handshake, PMKID attack (without capturing the handshake). Use strong passphrases (≥ 20 random characters).

WPA2-Enterprise (802.1X): EAP-based authentication (EAP-TLS, PEAP, EAP-TTLS). Each user has unique credentials (certificates or username/password). Requires a RADIUS server. Used in corporations. No shared passphrase: compromising one credential doesn't affect others.

WPA3-Personal: SAE (Simultaneous Authentication of Equals) Dragonfly handshake. Replaces PSK. Resistant to offline dictionary attacks (each attempt requires an online interaction). Forward secrecy.

WPA3-Enterprise: 192-bit security mode (CNSA suite). EAP-TLS with certificates mandatory.

Chapter 13

Advanced Topics: SDN, Cloud Networking, and Modern Architectures

13.1 Software Defined Networking (SDN)

Concept / Pattern

SDN Concept

Traditional networking: Control plane (routing decisions) and data plane (packet forwarding) are tightly coupled in each physical device.

SDN: Separates control plane from data plane.

- **SDN Controller:** Centralised brain. Computes routing decisions. Has a global view of the network topology. Communicates with data plane devices via southbound APIs (OpenFlow, gRPC).
- **Data Plane (forwarding devices):** Dumb switches/routers. Just follow flow tables programmed by the controller.
- **Northbound APIs:** Applications interact with the controller (e.g., request specific paths, query topology).

Benefits: Centralised management, programmable network behaviour, dynamic traffic engineering, easier automation, faster innovation.

Examples: Google B4 (WAN connecting data centres), OpenStack Neutron, Kubernetes CNI plugins.

13.1.1 OpenFlow

OpenFlow (ONF standard) is the first major southbound API:

- Controller maintains connections to all switches over TCP/TLS.
- Controller installs **flow entries** in each switch's flow table: match fields (src/dst IP/MAC, port, VLAN...) + actions (forward out port, drop, modify header, send to controller).
- Packet matching is like a hardware ACL with programmable actions.
- When a packet doesn't match any flow entry, it's sent to the controller (slow path) for a decision.

13.2 Container and Cloud Networking

13.2.1 Network Namespaces (Linux)

Network namespaces give each process group its own isolated network stack:

- Separate: interfaces, routing table, iptables rules, sockets.
- Containers (Docker, podman) use network namespaces for isolation.

- Each Docker container gets its own namespace with a virtual ethernet (**veth**) pair connecting it to the host namespace.

Example / Code

Working with Network Namespaces

```
# Create a network namespace
ip netns add myns

# Run a command in the namespace
ip netns exec myns ip addr

# Create a veth pair (two linked virtual interfaces)
ip link add veth0 type veth peer name veth1

# Move one end into the namespace
ip link set veth1 netns myns

# Configure both ends
ip addr add 10.0.0.1/24 dev veth0
ip link set veth0 up
ip netns exec myns ip addr add 10.0.0.2/24 dev veth1
ip netns exec myns ip link set veth1 up

# Test connectivity
ip netns exec myns ping 10.0.0.1
```

13.2.2 Docker Networking

Driver	Description
bridge	Default. Docker bridge (docker0). Containers on same bridge can communicate. NAT for external.
host	Container shares the host's network namespace. No isolation. Max performance.
none	No networking. Complete isolation.
overlay	Multi-host networking. VXLAN tunnels between hosts. Used in Docker Swarm.
macvlan	Container gets its own MAC address visible on physical network.

13.2.3 Kubernetes Networking

Kubernetes networking model:

- Every Pod gets its own IP address. Pods communicate directly without NAT.
- All Pods on all nodes can reach each other (flat network).
- Node-to-node connectivity via the CNI (Container Network Interface) plugin.
- **CNI plugins:** Flannel (overlay VXLAN), Calico (BGP routing, eBPF), Cilium (eBPF, advanced observability), WeaveNet, AWS VPC CNI.
- **Services:** ClusterIP (internal LB), NodePort (expose on host port), LoadBalancer (provision cloud LB), ExternalName (DNS alias).
- **kube-proxy:** Implements Service IPs using iptables or IPVS. Routes traffic to healthy Pod endpoints.
- **Ingress:** L7 HTTP routing. Ingress controllers: nginx, Traefik, AWS ALB Controller.
- **NetworkPolicy:** L3/L4 firewall for pods. “Allow pods with label app=frontend to reach pods with label app=backend on port 5432.”

13.3 eBPF: The Revolution in Network Programmability

eBPF (extended Berkeley Packet Filter) allows running sandboxed programs in the Linux kernel without changing kernel source code. In networking:

- **XDP (eXpress Data Path)**: Hook at the NIC driver level. Process packets before they enter the kernel networking stack. 25 Mpps per core vs 1 Mpps for iptables.
- **Traffic control (TC)**: Hook at the qdisc layer. Classify, modify, redirect packets.
- **Socket operations**: Intercept socket calls, redirect connections.
- **Cilium**: Kubernetes CNI using eBPF. Replaces iptables with eBPF programs. Microsegmentation, transparent encryption, load balancing at L4/L7.
- **Use cases**: DDoS mitigation (drop attack packets at wire speed), load balancing (Katran at Facebook: 1M pps per CPU), network observability (bpftrace, bcc tools), network policy enforcement.

Chapter 14

Network Diagnostic Tools

14.1 Essential Linux Networking Commands

Example / Code

```
# Connectivity

# ping: ICMP Echo Request/Reply. Test reachability and measure RTT.
ping -c 5 8.8.8.8 # 5 packets to Google DNS
ping -f 8.8.8.8 # Flood ping (root). Stress test.
ping6 2001:4860:4860::8888 # IPv6 ping

# traceroute: Map the path (and RTT per hop) to a destination.
# Sends packets with TTL=1,2,3... Each router sends ICMP Time Exceeded.
traceroute -n google.com # -n: skip DNS resolution (faster)
traceroute -T -p 443 google.com # TCP SYN (bypasses ICMP-blocking firewalls)
mtr --report --report-cycles 20 google.com # Continuous traceroute with stats

# DNS
dig google.com # A record (IPv4)
dig AAAA google.com # AAAA record (IPv6)
dig MX gmail.com # Mail exchange records
dig TXT google.com # TXT records (SPF, DKIM, etc.)
dig NS google.com # Name servers
dig @8.8.8.8 google.com # Query specific DNS server (bypass local)
dig +trace google.com # Full recursive trace from root
dig +short google.com # Just the IP
dig -x 8.8.8.8 # Reverse DNS (PTR) lookup
dig google.com +noall +answer # Show only the answer section

# Ports and Connections
ss -tlnp # All TCP listening sockets + PID
ss -tnp state established # All established TCP connections + PID
ss -tnp state time-wait # TIME_WAIT sockets (many = normal for busy server)
ss -tnp state close-wait # CLOSE_WAIT sockets (many = app bug)
ss -s # Summary statistics
ss -tnp 'dst :443' # Connections to port 443

# Check if a remote port is open
nc -zv 192.168.1.1 22 80 443 # -z: scan only, -v: verbose
timeout 3 bash -c '</dev/tcp/host/port' && echo "open" # Pure bash

# Traffic Capture
tcpdump -i eth0 # All traffic on eth0
tcpdump -i any port 53 # DNS on any interface
tcpdump -i eth0 'host 8.8.8.8 and port 443' # Filter by host and port
tcpdump -i eth0 -w /tmp/capture.pcap # Write to file (open in Wireshark)
tcpdump -i eth0 -X -s 0 port 80 # Hex+ASCII dump, full packet capture
tcpdump -i eth0 -nn 'tcp[tcpflags] & tcp-syn != 0' # Only SYN packets

# Interfaces and Routes
ip addr # Show interfaces and IP addresses
ip link # Show interface status (UP/DOWN, MAC)
ip route # Show routing table
ip route get 8.8.8.8 # Which route would be used for this IP?
ip neigh # ARP table (IP -> MAC mappings)
ip -6 route # IPv6 routing table

# Performance and Bandwidth
iperf3 -s # Start iperf3 server
iperf3 -c 192.168.1.1 -t 30 -P 4 # Test: 30 seconds, 4 parallel streams
iperf3 -c 192.168.1.1 -u -b 100M # UDP test at 100 Mbps

# Time a full HTTP request (DNS + connect + TLS + TTFB)
curl -o /dev/null -s -w "\
DNS lookup:      %{time_namelookup}s\n\
TCP connect:     %{time_connect}s\n\
TLS handshake:   %{time_appconnect}s\n\
TTFB:           %{time_starttransfer}s\n\
Total:          %{time_total}s\n" https://example.com

# TLS/Certificate
openssl s_client -connect google.com:443 -servername google.com
```

14.2 Reading tcpdump Output

Concept / Pattern

Dissecting a TCP 3-way Handshake in tcpdump

```
# tcpdump -nn 'host 93.184.216.34 and port 80'
12:00:01.000000 IP 192.168.1.5.54321 > 93.184.216.34.80: Flags [S],
    seq 100000, win 65535,
    options [mss 1460,sackOK,TS val 1234 ecr 0,nop,wscale 7], length 0

12:00:01.050000 IP 93.184.216.34.80 > 192.168.1.5.54321: Flags [S.],
    seq 200000, ack 100001, win 65535,
    options [mss 1460,sackOK,TS val 5678 ecr 1234,nop,wscale 9], length 0

12:00:01.050050 IP 192.168.1.5.54321 > 93.184.216.34.80: Flags [.],
    ack 200001, win 512, length 0

# Flag legend: [S]=SYN, [.] =ACK, [S.] =SYN+ACK, [F.] =FIN+ACK, [R]=RST, [P.] =PSH+ACK
# seq: sequence number, ack: acknowledgment number
# win: advertised window (in bytes; multiply by window scale factor)
# options: mss=1460, sackOK, timestamps (TS val/ecr), wscale (window scale)
# length: payload data length
```

Example / Code

Analysing TCP Retransmissions

```
# Capture and count retransmissions in real-time
tcpdump -nn -i eth0 'tcp[tcpflags] & tcp-syn == 0' 2>/dev/null \
| awk '/\[.\\.]/ {print}' | head -50

# With tshark (Wireshark CLI)
tshark -i eth0 -Y 'tcp.analysis.retransmission' -T fields \
-e frame.time -e ip.src -e ip.dst -e tcp.srcport -e tcp.dstport

# Monitor TCP stats (retransmits, errors)
watch -n 1 'netstat -s | grep -E "retransmit|fail|reset|error"'

# ss: Show TCP info including retransmit count per connection
ss -tin dst :443 | grep -E 'retrans|rtt'
```


Chapter 15

Networking Interview Masterclass

15.1 System Design and Networking Questions

Interview Corner

Q: What happens when you type `https://google.com` in a browser? (Complete answer)

1. **URL parsing:** Browser extracts scheme (`https`), host (`google.com`), path (`/`).
2. **HSTS check:** Browser checks its HSTS preload list / previous HSTS headers. If present, enforces HTTPS.
3. **DNS resolution:** Browser cache → OS cache (`/etc/hosts`) → stub resolver → recursive resolver → Root → `.com` TLD → `google.com` authoritative NS. Returns A record: `142.250.80.36`. Typically under 10ms with caching.
4. **TCP 3-way handshake:** SYN → SYN-ACK → ACK to `142.250.80.36:443`. RTT depends on distance.
5. **TLS 1.3 handshake:** ClientHello (key share, ciphers) → ServerHello + Certificate + Finished (all in 1 RTT). Browser verifies certificate chain (Root CA in trust store, cert not expired, domain in SAN, OCSP/CRL not revoked).
6. **HTTP/2 request:** GET / HTTP/2 with headers (Host: `www.google.com`, Accept, Accept-Encoding: `gzip`, etc.). (Note: `google.com` typically redirects to `www.google.com` with 301.)
7. **Server responds:** 200 OK with HTML, `gzip` compressed. Content-Security-Policy header set. Multiple cookies set.
8. **Browser renders:** Parses HTML, finds `<link>` (CSS), `<script>` (JS), `<img>`. Issues parallel requests over the same HTTP/2 connection (multiplexing).
9. **Subresource loading:** CSS, JS, images loaded. Some from CDN (`fonts.googleapis.com`, etc.). JavaScript executes, may make API calls.
10. **Ongoing:** Connection kept alive with HTTP/2 Keep-Alive. Browser uses WebSocket or Server-Sent Events if needed.

Interview Corner

Q: Design a system to handle 1 million concurrent WebSocket connections.

Key challenges:

- 1M file descriptors: Increase OS limits (`ulimit -n 1048576, /proc/sys/fs/file-max`).
- 1M TCP connections: Each needs kernel memory for socket buffer, TCP control blocks (~4-10 KB each = 4-10 GB RAM just for buffers).
- High connection churn: Many connections opening/closing.

Architecture:

1. **Connection gateway tier:** Dedicated servers running epoll-based WebSocket servers. Each can handle 100K–1M connections with tuning. Use 10–100 servers. These are pure connection holders (stateless gateways).
2. **Message bus:** Kafka or Redis Pub/Sub to distribute messages from business logic to the correct gateway server. Gateway subscribes to channels for its connected clients.
3. **Client-to-server mapping:** Redis stores `{client_id → gateway_server_id}`. When business logic needs to push to client, looks up gateway, publishes to it.
4. **Load balancing:** L4 LB with consistent hashing (route client to same gateway). Or L7 with sticky sessions.
5. **Heartbeats:** Client sends ping every 30s. Server sends pong. Close connection after 60s of silence.
6. **Horizontal scaling:** Add more gateways; update the routing layer.

OS tuning: `net.ipv4.ip_local_port_range = 1024 65535, net.core.somaxconn = 65535, net.ipv4.tcp_max_syn_backlog = 65535, vm.swappiness = 0.`

Interview Corner

Q: How would you debug a slow HTTP request?

1. **Measure each phase** with curl timing output (DNS, TCP connect, TLS, TTFB, download).
2. **DNS slow?:** `dig +time=1` to check resolver. Use `@8.8.8.8` to bypass. Check TTL.
3. **TCP connect slow?:** High RTT = far from server (use `traceroute + mtr`). Packet loss causes retransmission.
4. **TLS slow?:** TLS version, cipher suite, OCSP latency (check for OCSP stapling). Certificate chain too long.
5. **TTFB (Time To First Byte) slow?:** Server-side processing. Profile the application. Check database queries, cache misses, lock contention.
6. **Download slow?:** Bandwidth issue. Check `rwnd, cwnd`. Use Wireshark to check if TCP window is scaling. Check for packet loss causing retransmissions.
7. **Tools:** Wireshark (full packet analysis), `tcpdump`, `ss` (TCP state and stats), `strace` on the server (system call timing), application profiler (`perf`, `go pprof`, Java Flight Recorder).

15.2 TCP/IP Deep Questions

Interview Corner**Q: Why does TCP use a 3-way handshake instead of 2-way?**

Two messages cannot prove full bidirectional connectivity:

- **SYN:** Client proves it can *send*.
- **SYN-ACK:** Server proves it can *receive and send*. But client doesn't yet know if server received the SYN properly.
- **ACK:** Client proves it can *receive* (echoes server's ISN + 1). Now both sides know the connection is bidirectional.

The 3rd message (client's ACK) also serves to tell the server that the client's ISN was received (via the ACK number = server_ISN + 1).

Q: What is TCP half-open?

A TCP half-open connection is one where one side thinks the connection is ESTABLISHED but the other side is not (e.g., the other side crashed and rebooted, losing its state). When the “alive” side sends data, the rebooted side responds with RST. Network scanners like nmap can create half-open connections by sending SYN and never responding to SYN-ACK (never completing the handshake), useful for stealth scanning.

Interview Corner**Q: What is TCP buffer bloat and how does it affect latency?**

Buffer bloat: ISPs and home routers have very large buffers (to prevent packet loss). When a link is congested, these buffers fill up instead of dropping packets immediately. TCP interprets “no packet loss” as “no congestion” and keeps sending. The large buffers can cause hundreds of milliseconds of queuing delay, making real-time applications (VoIP, gaming) terrible even though throughput is fine.

Measurement: `ping -f` during a download. If ping RTT spikes massively (>100ms) while downloading, you have buffer bloat.

Solutions:

- **FQ-CoDel / CAKE:** Smart queue management (SQM) algorithms. Detect and manage buffer bloat by actively dropping/marketing packets when queuing delay exceeds a threshold (typically 5ms). Available in OpenWrt, modern Linux. FQ-CoDel is the Linux default.
- **BBR:** Google's congestion control measures actual bottleneck bandwidth and RTT, avoiding over-filling buffers.
- **ECN (Explicit Congestion Notification):** Routers mark packets (set CE bits in IP header) instead of dropping when approaching congestion. TCP senders reduce rate. Requires ECN support at both endpoints and routers.

15.3 Security Interview Questions

Interview Corner

Q: Explain the difference between HTTP 401 and 403.

401 Unauthorized: The server doesn't know *who* you are. Authentication is missing or invalid. Include a `WWW-Authenticate` header to tell the client how to authenticate. Example: no Bearer token in the request, or the token is expired/invalid.

403 Forbidden: The server knows who you are (authenticated), but you don't have permission to access this resource. Example: a regular user trying to access an admin endpoint. No point in re-authenticating.

Common confusion: APIs sometimes return 401 even when the client is authenticated but unauthorised (to avoid leaking information about which resources exist). This is a security consideration, not a protocol requirement.

Q: What is SSRF (Server-Side Request Forgery)?

Attack: An attacker makes the server perform HTTP requests to internal/metadata services on their behalf. If a web app accepts a URL and fetches it ("preview this URL"), an attacker can input `http://169.254.169.254/` (AWS/GCP instance metadata service) to steal IAM credentials, or `http://internal-db:5432/` to probe internal services.

Mitigations: Allowlist of allowed URL schemes/hosts; block private IP ranges (10.x, 172.16.x, 192.168.x, 169.254.x, 127.x) at the application level; use DNS rebinding protection; don't follow redirects to internal hosts; use a dedicated outbound proxy that enforces allowlists.

Interview Corner

Q: What is HTTPS and does it protect you completely?

HTTPS = HTTP over TLS. Provides: confidentiality (encrypted data), authentication (server certificate), integrity (AEAD prevents tampering).

What HTTPS does NOT protect against:

- **Application-layer attacks:** XSS, SQLi, CSRF, IDOR — HTTPS encrypts the transport but doesn't validate what's inside the payload.
- **Compromised endpoints:** If the server is hacked or the client has malware, TLS provides no protection at the endpoints.
- **Metadata leakage:** DNS queries reveal the domain (use DoH/DoT to encrypt). IP address reveals the server. SNI (Server Name Indication) in the ClientHello reveals the hostname (use ECH to encrypt).
- **Traffic analysis:** Even encrypted, packet size and timing patterns can reveal information (website fingerprinting, keystroke timing).
- **Compromised CA:** If a CA is hacked (DigiNotar 2011, Symantec 2017), fraudulent certs can enable MITM. Certificate Transparency and HPKP (deprecated) help detect this.
- **0-RTT replay attacks:** TLS 1.3's 0-RTT data can be replayed if not handled carefully.

Interview Corner

Q: DNS TTL is 0. A website is slow. CNAME vs A record: which is faster?

CNAME vs A at the client:

`www.example.com` CNAME `example.com` → resolver first resolves CNAME to `example.com`, then resolves `example.com`'s A record. Two DNS lookups if not cached. Slightly more latency.

A record: Single DNS lookup.

In practice: If the intermediate records are cached, CNAME adds negligible overhead. But in high-performance scenarios or with TTL=0, extra latency from CNAME resolution is measurable (especially with 2–3 levels of CNAME chaining). Use ALIAS/ANAME at the zone apex when you need CNAME-like behaviour without the double lookup cost.

TTL=0 issue: Every page load requires a fresh DNS lookup. This adds 50–200ms per DNS lookup (if resolver is far or cache misses). Set a reasonable TTL (300–3600s). The website slowness is likely due to DNS latency from TTL=0.

Interview Corner

Q: Two processes try to bind to port 80 on the same machine. What happens?

Without `SO_REUSEPORT`: The second `bind()` call returns `EADDRINUSE`. Only one process can listen on a port at a time.

With `SO_REUSEPORT`: Both processes can bind to port 80. The OS distributes incoming connections between them (load balancing). This is how nginx achieves multi-worker parallel accept without a single lock. Each worker process independently calls `accept()` on its own socket. Connections are distributed by 4-tuple hash.

Note: Both processes must set `SO_REUSEPORT` before binding. If one sets it and the other doesn't, it still fails.

Docker/containers: Port conflicts happen when two containers try to expose the same host port. Docker returns an error. Use different host ports.

Interview Corner

Q: How does HTTPS load balancing work? Does the LB decrypt the traffic?

TLS Pass-through (L4 LB): The LB forwards encrypted TCP bytes without decrypting. The backend server terminates TLS. The LB cannot see HTTP headers, URLs, or cookies — it can only route based on IP/port. Very fast, no certificate on the LB.

TLS Termination (L7 LB): The LB terminates TLS (needs the server's private key/certificate). It decrypts the traffic, reads HTTP headers/URL/cookies, makes routing decisions, then optionally re-encrypts for the backend (TLS re-origination). Enables: HTTP-based routing, header manipulation, WAF, request buffering, compression.

TLS Re-origination: After L7 LB decrypts, it establishes a new TLS connection to the backend. Used for end-to-end encryption in regulated environments (PCI-DSS, HIPAA). Adds CPU overhead at the LB.

X-Forwarded-For: When the LB terminates TLS and connects to backends, the backend only sees the LB's IP, not the original client IP. The LB adds `X-Forwarded-For: original-client-ip` (or `Forwarded: for=...` per RFC 7239) to pass the original client IP to backends.